

Hardware Description Languages

4

4.1 INTRODUCTION

Thus far, we have focused on designing combinational and sequential digital circuits at the schematic level. The process of finding an efficient set of logic gates to perform a given function is labor intensive and error prone, requiring manual simplification of truth tables or Boolean equations and manual translation of finite state machines (FSMs) into gates. In the 1990's, designers discovered that they were far more productive if they worked at a higher level of abstraction, specifying just the logical function and allowing a *computer-aided design* (CAD) tool to produce the optimized gates. The specifications are generally given in a *hardware description language* (HDL). The two leading hardware description languages are *SystemVerilog* and *VHDL*.

SystemVerilog and VHDL are built on similar principles but have different syntax. Discussion of these languages in this chapter is divided into two columns for literal side-by-side comparison, with SystemVerilog on the left and VHDL on the right. When you read the chapter for the first time, focus on one language or the other. Once you know one, you'll quickly master the other if you need it.

Subsequent chapters show hardware in both schematic and HDL form. If you choose to skip this chapter and not learn one of the HDLs, you will still be able to master the principles of computer organization from the schematics. However, the vast majority of commercial systems are now built using HDLs rather than schematics. If you expect to do digital design at any point in your professional life, we urge you to learn one of the HDLs.

4.1.1 Modules

A block of hardware with inputs and outputs is called a *module*. An AND gate, a multiplexer, and a priority circuit are all examples of hardware modules. The two general styles for describing module functionality are

- 4.1 Introduction
- 4.2 Combinational Logic
- 4.3 Structural Modeling
- 4.4 Sequential Logic
- 4.5 More Combinational Logic
- 4.6 Finite State Machines
- 4.7 Data Types*
- 4.8 Parameterized Modules*
- 4.9 Testbenches
- 4.10 Summary
- Exercises
- Interview Questions

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Some people still use Verilog, the original incarnation of SystemVerilog. So, we provide a version of this chapter describing Verilog on the companion website (see the Preface).

behavioral and *structural*. Behavioral models describe what a module does. Structural models describe how a module is built from simpler pieces; it is an application of hierarchy. The SystemVerilog and VHDL code in [HDL Example 4.1](#) illustrate behavioral descriptions of a module that computes the Boolean function from Example 2.6, $y = \bar{a}\bar{b}\bar{c} + a\bar{b}\bar{c} + a\bar{b}c$. In both languages, the module is named `sillyfunction` and has three inputs, `a`, `b`, and `c`, and one output, `y`.

HDL Example 4.1 COMBINATIONAL LOGIC

SystemVerilog

```
module sillyfunction(input logic a, b, c,
                    output logic y);

    assign y = ~a & ~b & ~c |
              a & ~b & ~c |
              a & ~b & c;

endmodule
```

A SystemVerilog module begins with the module name and a listing of the inputs and outputs. The `assign` statement describes combinational logic. `~` indicates NOT, `&` indicates AND, and `|` indicates OR.

`logic` signals such as the inputs and outputs are Boolean variables (0 or 1). They may also have floating and undefined values, as discussed in [Section 4.2.8](#).

The `logic` type was introduced in SystemVerilog. It supersedes the `reg` type, which was a perennial source of confusion in Verilog. `logic` should be used everywhere except on signals with multiple drivers. Signals with multiple drivers are called *nets* and will be explained in [Section 4.7](#).

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sillyfunction is
    port(a, b, c: in STD_LOGIC;
         y: out STD_LOGIC);
end;

architecture synth of sillyfunction is
begin
    y <= (not a and not b and not c) or
         (a and not b and not c) or
         (a and not b and c);
end;
```

VHDL code has three parts: the library use clause, the entity declaration, and the architecture body. The library use clause will be discussed in [Section 4.7.2](#). The entity declaration lists the module name and its inputs and outputs. The architecture body defines what the module does.

VHDL signals, such as inputs and outputs, must have a *type declaration*. Digital signals should be declared to be `STD_LOGIC` type. `STD_LOGIC` signals can have a value of '0' or '1' as well as floating and undefined values that will be described in [Section 4.2.8](#). The `STD_LOGIC` type is defined in the `IEEE.STD_LOGIC_1164` library, which is why the library must be used.

VHDL lacks a good default order of operations between AND and OR, so Boolean equations should be parenthesized.

A module, as you might expect, is a good application of modularity. It has a well-defined interface, consisting of its inputs and outputs, and it performs a specific function. The particular way in which it is coded is unimportant to others that might use the module, as long as it performs its function.

4.1.2 Language Origins

Universities are almost evenly split on which of these languages is taught in a first course. Industry is trending toward SystemVerilog, but many companies still use VHDL, so many designers need to be fluent in both.

SystemVerilog

Verilog was developed by Gateway Design Automation as a proprietary language for logic simulation in 1984. Gateway was acquired by Cadence in 1989 and Verilog was made an open standard in 1990, under the control of Open Verilog International. The language became an IEEE standard¹ in 1995. The language was extended in 2005 to streamline idiosyncrasies and to better support modeling and verification of systems. These extensions have been merged into a single language standard, which is now called SystemVerilog (IEEE STD 1800-2009). SystemVerilog file names normally end in `.sv`.

VHDL

VHDL is an acronym for the *VHSIC Hardware Description Language*. VHSIC is, in turn, an acronym for the *Very High Speed Integrated Circuits* program of the US Department of Defense.

VHDL was originally developed in 1981 by the Department of Defense to describe the structure and function of hardware. Its roots draw from the Ada programming language. The language was first envisioned for documentation but was quickly adopted for simulation and synthesis. The IEEE standardized it in 1987 and has updated the standard several times since. This chapter is based on the 2008 revision of the VHDL standard (IEEE STD 1076-2008), which streamlines the language in a variety of ways. To use VHDL 2008 in ModelSim, you may need to set `VHDL93 = 2008` in the `modelsim.ini` configuration file. VHDL file names normally end in `.vhd`.

Compared to SystemVerilog, VHDL is more verbose and cumbersome, as you might expect of a language developed by committee.

Both languages are fully capable of describing any hardware system, and both have their quirks. The best language to use is the one that is already being used at your site or the one that your customers demand. Most CAD tools today allow the two languages to be mixed so that different modules can be described in different languages.

4.1.3 Simulation and Synthesis

The two major purposes of HDLs are logic *simulation* and *synthesis*. During simulation, inputs are applied to a module, and the outputs are checked to verify that the module operates correctly. During synthesis, the textual description of a module is transformed into logic gates.

Simulation

Humans routinely make mistakes. Such errors in hardware designs are called *bugs*. Eliminating the bugs from a digital system is obviously important, especially when customers are paying money and lives depend on the correct operation. Testing a system in the laboratory is time-consuming. Discovering the cause of errors in the lab can be extremely difficult because only signals routed to the chip pins can be observed.

The term “bug” predates the invention of the computer. Thomas Edison called the “little faults and difficulties” with his inventions “bugs” in 1878.

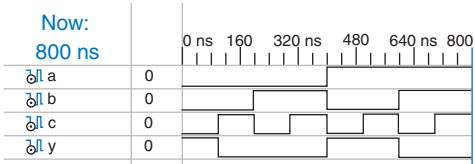
The first real computer bug was a moth, which got caught between the relays of the Harvard Mark II electro-mechanical computer in 1947. It was found by Grace Hopper, who logged the incident, along with the moth itself and the comment “first actual case of bug being found.”



Source: Notebook entry courtesy Naval Historical Center, US Navy; photo No. NII 96566-KN

¹The Institute of Electrical and Electronics Engineers (IEEE) is a professional society responsible for many computing standards, including Wi-Fi (802.11), Ethernet (802.3), and floating-point numbers (754).

Figure 4.1 Simulation waveforms



There is no way to directly observe what is happening inside a chip. Correcting errors after the system is built can be devastatingly expensive. For example, correcting a mistake in a cutting-edge integrated circuit costs more than a million dollars and takes several months. Intel’s infamous FDIV (floating point division) bug in the Pentium processor forced the company to recall chips after they had shipped at a total cost of \$475 million in 1984. Logic simulation is essential to test a system before it is built.

Figure 4.1 shows waveforms from a simulation² of the previous `sillyfunction` module, demonstrating that the module works correctly. `y` is TRUE when `a`, `b`, and `c` are 000, 100, or 101, as specified by the Boolean equation.

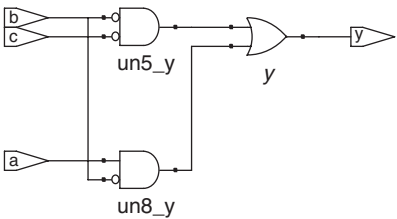
Synthesis

Logic synthesis transforms HDL code into a *netlist* describing the hardware (e.g., the logic gates and the wires connecting them). The logic synthesizer might perform optimizations to reduce the amount of hardware required. The netlist may be a text file, or it may be drawn as a schematic to help visualize the circuit. Figure 4.2 shows the results of synthesizing the `sillyfunction` module.³ Notice how the three three-input AND gates are simplified into two two-input AND gates, as we discovered in Example 2.6 using Boolean algebra.

Circuit descriptions in HDL resemble code in a programming language. However, you must remember that the code is intended to represent hardware. SystemVerilog and VHDL are rich languages with many commands.

The synthesis tool labels each of the synthesized gates. In Figure 4.2, they are `un5_y`, `un8_y`, and `y`.

Figure 4.2 Synthesized circuit



²The simulations in this chapter were performed with the ModelSim PE Student Edition Version 10.3c. ModelSim was selected because it is used commercially, yet a student version with a capacity of 10,000 lines of code is freely available.

³Throughout this chapter, synthesis was performed using Synplify Premier from Synopsys. However, many synthesis tools exist, such as those included in Vivado and Quartus, the freely available Xilinx and Intel design tools for synthesizing HDL to field programmable gate arrays (see Section 5.6.2).

Not all of these commands can be synthesized into hardware. For example, a command to print results on the screen during simulation does not translate into hardware. Because our primary interest is to build hardware, we will emphasize a *synthesizable subset* of the languages. Specifically, we will divide HDL code into *synthesizable* modules and a *testbench*. The synthesizable modules describe the hardware. The testbench contains code to apply inputs to a module, check whether the output results are correct, and print discrepancies between expected and actual outputs. Testbench code is intended only for simulation and cannot be synthesized.

One of the most common mistakes for beginners is to think of HDL as a computer program rather than as a shorthand for describing digital hardware. If you don't know approximately what hardware your HDL should synthesize into, you probably won't like what you get. You might create far more hardware than is necessary, or you might write code that simulates correctly but cannot be implemented in hardware. Instead, think of your system in terms of blocks of combinational logic, registers, and finite state machines. Sketch these blocks on paper and show how they are connected before you start writing code.

In our experience, the best way to learn an HDL is by example. HDLs have specific ways of describing various classes of logic; these ways are called *idioms*. This chapter will teach you how to write the proper HDL idioms for each type of block and then how to put the blocks together to produce a working system. When you need to describe a particular kind of hardware, look for a similar example and adapt it to your purpose. We do not attempt to rigorously define all the syntax of the HDLs, because that is deathly boring and it tends to encourage thinking of HDLs as programming languages, not shorthand for hardware. The IEEE SystemVerilog and VHDL specifications, and numerous dry but exhaustive textbooks, contain all of the details, should you find yourself needing more information on a particular topic. (See the Further Readings section at the back of the book.)

4.2 COMBINATIONAL LOGIC

Recall that we are disciplining ourselves to design synchronous sequential circuits, which consist of combinational logic and registers. The outputs of combinational logic depend only on the current inputs. This section describes how to write behavioral models of combinational logic with HDLs.

4.2.1 Bitwise Operators

Bitwise operators act on single-bit signals or on multibit busses. For example, the `inv` module in [HDL Example 4.2](#) describes four inverters connected to 4-bit busses.

HDL Example 4.2 INVERTERS

SystemVerilog

```
module inv(input  logic [3:0] a,
          output logic [3:0] y);

    assign y = ~a;
endmodule
```

`a[3:0]` represents a 4-bit bus. The bits, from most significant to least significant, are `a[3]`, `a[2]`, `a[1]`, and `a[0]`. This is called *little-endian* order, because the least significant bit has the smallest bit number. We could have named the bus `a[4:1]`, in which case `a[4]` would have been the most significant. Or we could have used `a[0:3]`, in which case the bits, from most significant to least significant, would be `a[0]`, `a[1]`, `a[2]`, and `a[3]`. This is called *big-endian* order.

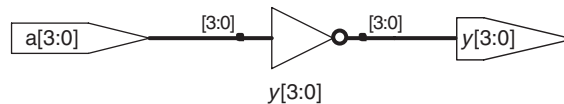
VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity inv is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of inv is
begin
    y <= not a;
end;
```

VHDL uses `STD_LOGIC_VECTOR` to indicate busses of `STD_LOGIC`. `STD_LOGIC_VECTOR(3 downto 0)` represents a 4-bit bus. The bits, from most significant to least significant, are `a(3)`, `a(2)`, `a(1)`, and `a(0)`. This is called *little-endian* order, because the least significant bit has the smallest bit number. We could have declared the bus to be `STD_LOGIC_VECTOR(4 downto 1)`, in which case bit 4 would have been the most significant. Or we could have written `STD_LOGIC_VECTOR(0 to 3)`, in which case the bits, from most significant to least significant, would be `a(0)`, `a(1)`, `a(2)`, and `a(3)`. This is called *big-endian* order.

Figure 4.3 `inv` synthesized circuit

The endianness of a bus is purely arbitrary. (See the sidebar in [Section 6.6.1](#) for the origin of the term.) Indeed, endianness is also irrelevant to this example because a bank of inverters doesn't care what the order of the bits are. Endianness matters only for operators, such as addition, where the sum of one column carries over into the next. Either ordering is acceptable as long as it is used consistently. We will consistently use the little-endian order, `[N-1:0]` in SystemVerilog and `(N-1 downto 0)` in VHDL, for an N -bit bus.

After each code example in this chapter is a schematic produced from the SystemVerilog code by the Synplify Premier synthesis tool. [Figure 4.3](#) shows that the `inv` module synthesizes to a bank of four inverters, indicated by the inverter symbol labeled `y[3:0]`. The bank of inverters connects to 4-bit input and output busses. Similar hardware is produced from the synthesized VHDL code.

The `gates` module in [HDL Example 4.3](#) demonstrates bitwise operations acting on 4-bit busses for other basic logic functions.

HDL Example 4.3 LOGIC GATES**SystemVerilog**

```

module gates(input logic [3:0] a, b,
             output logic [3:0] y1, y2,
                    y3, y4, y5);

/* five different two-input logic
   gates acting on 4-bit busses */
assign y1 = a & b;    // AND
assign y2 = a | b;    // OR
assign y3 = a ^ b;    // XOR
assign y4 = ~(a & b); // NAND
assign y5 = ~(a | b); // NOR
endmodule

```

$\sim$, $\wedge$, and $|$ are examples of SystemVerilog *operators*, whereas a , b , and $y1$ are *operands*. A combination of operators and operands, such as $a \& b$, or $\sim(a | b)$, is called an *expression*. A complete command such as `assign y4 =  $\sim(a \& b)$ ;` is called a *statement*.

`assign out = in1 op in2;` is called a *continuous assignment statement*. Continuous assignment statements end with a semicolon. Whenever the inputs on the right side of the $=$ in a continuous assignment statement change, the output on the left side is recomputed. Thus, continuous assignment statements describe combinational logic.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity gates is
port(a, b: in  STD_LOGIC_VECTOR(3 downto 0);
     y1, y2, y3, y4,
     y5: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of gates is
begin
-- five different two-input logic gates
-- acting on 4-bit busses
y1 <= a and b;
y2 <= a or b;
y3 <= a xor b;
y4 <= a nand b;
y5 <= a nor b;
end;

```

`not`, `xor`, and `or` are examples of VHDL *operators*, whereas a , b , and $y1$ are *operands*. A combination of operators and operands, such as a and b , or a nor b , is called an *expression*. A complete command such as `y4 <= a nand b;` is called a *statement*.

`out <= in1 op in2;` is called a *concurrent signal assignment statement*. VHDL assignment statements end with a semicolon. Whenever the inputs on the right side of the $<=$ in a concurrent signal assignment statement change, the output on the left side is recomputed. Thus, concurrent signal assignment statements describe combinational logic.

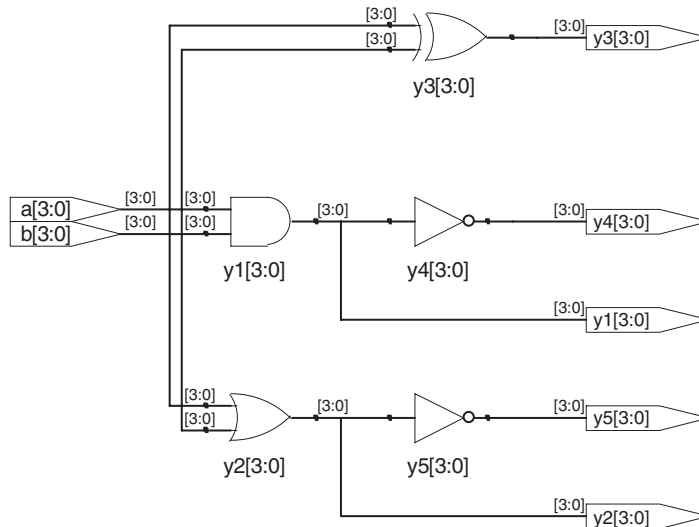


Figure 4.4 gates synthesized circuit

4.2.2 Comments and White Space

The `gates` example showed how to format comments. SystemVerilog and VHDL are not picky about the use of white space (i.e., spaces, tabs, and line breaks). Nevertheless, proper indenting and use of blank lines is helpful to make nontrivial designs readable. Be consistent in your use of capitalization and underscores in signal and module names. This text uses all lower case. Module and signal names must not begin with a digit.

SystemVerilog	VHDL
SystemVerilog comments are just like those in C or Java. Comments beginning with <code>/*</code> continue, possibly across multiple lines, to the next <code>*/</code>. Comments beginning with <code>//</code> continue to the end of the line. SystemVerilog is case-sensitive. <code>y1</code> and <code>Y1</code> are different signals in SystemVerilog. However, it is confusing to use multiple signals that differ only in case.	Comments beginning with <code>/*</code> continue, possibly across multiple lines, to the next <code>*/</code>. Comments beginning with <code>--</code> continue to the end of the line. VHDL is not case-sensitive. <code>y1</code> and <code>Y1</code> are the same signal in VHDL. However, other tools that may read your file might be case-sensitive, leading to nasty bugs if you blithely mix upper and lower case.

4.2.3 Reduction Operators

Reduction operators imply a multiple-input gate acting on a single bus. [HDL Example 4.4](#) describes an eight-input AND gate with inputs `a7`, `a6`, . . . , `a0`. Analogous reduction operators exist for OR, XOR, NAND, NOR, and XNOR gates. Recall that a multiple-input XOR performs parity, returning TRUE if an odd number of inputs are TRUE.

HDL Example 4.4 EIGHT-INPUT AND

SystemVerilog	VHDL
<pre>module and8(input logic[7:0] a, output logic y); assign y=&a; // &a is much easier to write than // assign y=a[7] & a[6] & a[5] & a[4] & // a[3] & a[2] & a[1] & a[0]; endmodule</pre>	<pre>library IEEE; use IEEE.STD_LOGIC_1164.all; entity and8 is port(a: in STD_LOGIC_VECTOR(7 downto 0); y: out STD_LOGIC); end; architecture synth of and8 is begin y <= and a; -- and a is much easier to write than -- y <= a(7) and a(6) and a(5) and a(4) and -- a(3) and a(2) and a(1) and a(0); end;</pre>

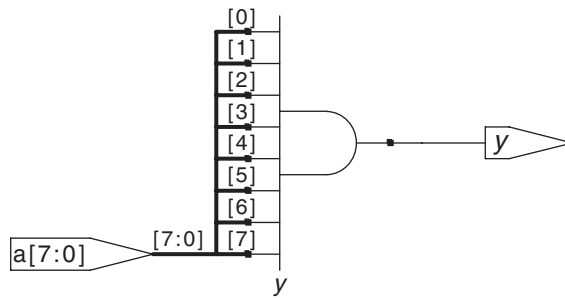


Figure 4.5 and8 synthesized circuit

4.2.4 Conditional Assignment

Conditional assignments select the output from among alternatives based on an input called the *condition*. HDL Example 4.5 illustrates a 2:1 multiplexer using conditional assignment.

HDL Example 4.5 2:1 MULTIPLEXER

SystemVerilog

The *conditional operator* `?:` chooses, based on a first expression, between a second and third expression. The first expression is called the *condition*. If the condition is 1, the operator chooses the second expression. If the condition is 0, the operator chooses the third expression.

`?:` is especially useful for describing a multiplexer because, based on the first input, it selects between two others. The following code demonstrates the idiom for a 2:1 multiplexer with 4-bit inputs and outputs using the conditional operator.

```
module mux2(input  logic [3:0] d0, d1,
            input  logic   s,
            output logic [3:0] y);

    assign y = s ? d1 : d0;
endmodule
```

If `s` is 1, then `y = d1`. If `s` is 0, then `y = d0`.

`?:` is also called a *ternary operator* because it takes three inputs. It is used for the same purpose in the C and Java programming languages.

VHDL

Conditional signal assignments perform different operations, depending on some condition. They are especially useful for describing a multiplexer. For example, a 2:1 multiplexer can use conditional signal assignment to select one of two 4-bit inputs.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
    port(d0, d1: in  STD_LOGIC_VECTOR(3 downto 0);
         s:      in  STD_LOGIC;
         y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of mux2 is
begin
    y <= d1 when s else d0;
end;
```

The conditional signal assignment sets `y` to `d1` if `s` is 1. Otherwise, it sets `y` to `d0`. Note that prior to the 2008 revision of VHDL, one had to write `when s = '1'` rather than `when s`.

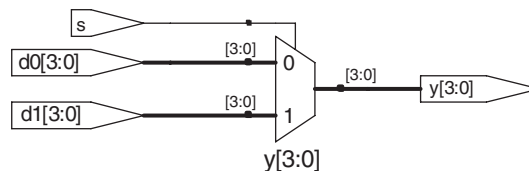


Figure 4.6 mux2 synthesized circuit

HDL Example 4.6 shows a 4:1 multiplexer based on the same principle as the 2:1 multiplexer in **HDL Example 4.5**. **Figure 4.7** shows the schematic for the 4:1 multiplexer produced by the synthesis tool. The software uses a different multiplexer symbol than this text has shown so far. The multiplexer has multiple data (d) and one-hot enable (e) inputs. When one of the enables is asserted, the associated data is passed to the output. For example, when $s[1] = s[0] = 0$, the bottom AND gate, `un1_s_5`, produces a 1, enabling the bottom input of the multiplexer and causing it to select `d0[3:0]`.

HDL Example 4.6 4:1 MULTIPLEXER

SystemVerilog

A 4:1 multiplexer can select one of four inputs, using nested conditional operators.

```
module mux4(input    logic [3:0] d0, d1, d2, d3,
            input    logic [1:0] s,
            output   logic [3:0] y);

    assign y = s[1] ? (s[0] ? d3 : d2)
                : (s[0] ? d1 : d0);
endmodule
```

If $s[1]$ is 1, then the multiplexer chooses the first expression, $(s[0] ? d3 : d2)$. This expression, in turn, chooses either $d3$ or $d2$ based on $s[0]$ ($y = d3$ if $s[0]$ is 1 and $d2$ if $s[0]$ is 0). If $s[1]$ is 0, then the multiplexer similarly chooses the second expression, which gives either $d1$ or $d0$ based on $s[0]$.

VHDL

A 4:1 multiplexer can select one of four inputs, using multiple `else` clauses in the conditional signal assignment.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4 is
    port(d0, d1,
          d2, d3: in  STD_LOGIC_VECTOR(3 downto 0);
          s:      in  STD_LOGIC_VECTOR(1 downto 0);
          y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth1 of mux4 is
begin
    y <= d0 when s = "00" else
          d1 when s = "01" else
          d2 when s = "10" else
          d3;
end;
```

VHDL also supports *selected signal assignment statements* to provide a shorthand when selecting from one of several possibilities. This is analogous to using a `switch/case` statement in place of multiple `if/else` statements in some programming languages. The 4:1 multiplexer can be rewritten with selected signal assignment as follows:

```
architecture synth2 of mux4 is
begin
    with s select y <=
        d0 when "00",
        d1 when "01",
        d2 when "10",
        d3 when others;
end;
```

4.2.5 Internal Variables

Often it is convenient to break a complex function into intermediate steps. For example, a full adder, which will be described in **Section 5.2.1**,

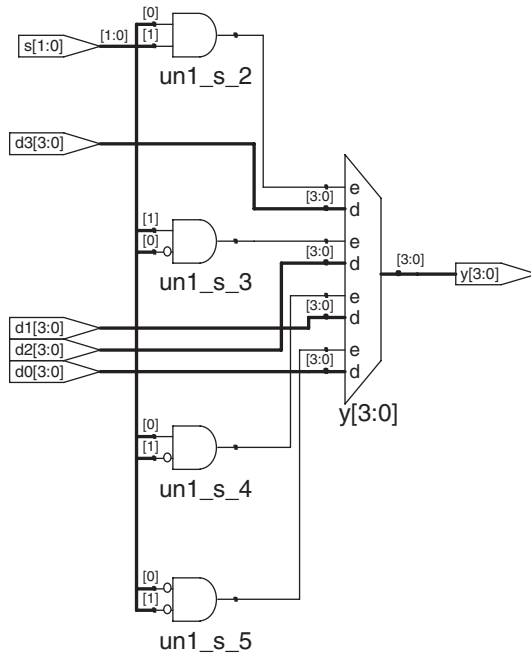


Figure 4.7 mux4 synthesized circuit

is a circuit with three inputs and two outputs defined by the following equations:

$$\begin{aligned} S &= A \oplus B \oplus C_{\text{in}} \\ C_{\text{out}} &= AB + AC_{\text{in}} + BC_{\text{in}} \end{aligned} \quad (4.1)$$

If we define intermediate signals, P and G ,

$$\begin{aligned} P &= A \oplus B \\ G &= AB \end{aligned} \quad (4.2)$$

we can rewrite the full adder as follows:

$$\begin{aligned} S &= P \oplus C_{\text{in}} \\ C_{\text{out}} &= G + PC_{\text{in}} \end{aligned} \quad (4.3)$$

P and G are called *internal variables* because they are neither inputs nor outputs; rather, they are used only internal to the module. They are similar to local variables in programming languages. [HDL Example 4.7](#) shows how they are used in HDLs.

HDL assignment statements (`assign` in SystemVerilog and `<=` in VHDL) take place concurrently. This is different from conventional programming languages, such as C or Java, in which statements are evaluated in the order in which they are written. In a conventional language, it is

Check this by filling out the truth table to convince yourself it is correct.

HDL Example 4.7 FULL ADDER**SystemVerilog**

In SystemVerilog, internal signals are usually declared as `logic`.

```
module fulladder(input logic a, b, cin,
                output logic s, cout);

    logic p, g;

    assign p = a ^ b;
    assign g = a & b;

    assign s = p ^ cin;
    assign cout = g | (p & cin);
endmodule
```

VHDL

In VHDL, *signals* are used to represent internal variables whose values are defined by *concurrent signal assignment statements*, such as `p <= a xor b`;

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity fulladder is
    port(a, b, cin: in  STD_LOGIC;
         s, cout: out STD_LOGIC);
end;

architecture synth of fulladder is
    signal p, g: STD_LOGIC;
begin
    p <= a xor b;
    g <= a and b;

    s <= p xor cin;
    cout <= g or (p and cin);
end;
```

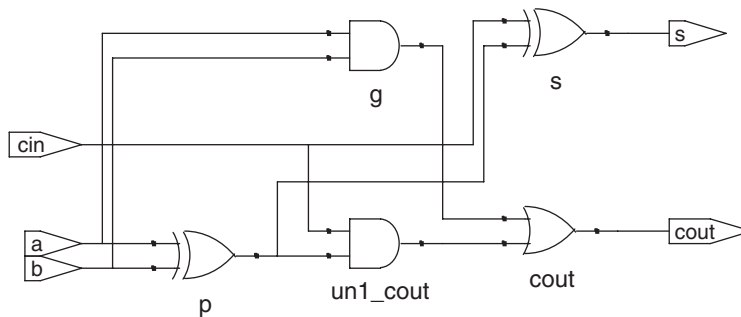


Figure 4.8 fulladder synthesized circuit

important that $S = P \oplus C_{in}$ comes after $P = A \oplus B$ because statements are executed sequentially. In an HDL, the order does not matter. Like hardware, HDL assignment statements are evaluated whenever the inputs, signals on the right-hand side, change their value regardless of the order in which the assignment statements appear in a module.

4.2.6 Precedence

Notice that we parenthesized the `cout` computation in [HDL Example 4.7](#) to define the order of operations as $C_{out} = G + (P \cdot C_{in})$ rather than $C_{out} = (G + P) \cdot C_{in}$. If we had not used parentheses, the default operation order would be defined by the language. [HDL Example 4.8](#)

HDL Example 4.8 OPERATOR PRECEDENCE

SystemVerilog	VHDL																																								
Table 4.1 SystemVerilog operator precedence <table><tr><th>Op</th><th>Meaning</th></tr><tr><td>H i g h e s t</td><td>~ NOT</td></tr><tr><td></td><td>*, /, % MUL, DIV, MOD</td></tr><tr><td></td><td>+, - PLUS, MINUS</td></tr><tr><td></td><td><<, >> Logical Left/Right Shift</td></tr><tr><td></td><td><<<, >>> Arithmetic Left/Right Shift</td></tr><tr><td></td><td><, <=, >, >= Relative Comparison</td></tr><tr><td></td><td>==, != Equality Comparison</td></tr><tr><td>L o w e s t</td><td>&, ~& AND, NAND</td></tr><tr><td></td><td>^, ~^ XOR, XNOR</td></tr><tr><td></td><td> , ~ OR, NOR</td></tr><tr><td></td><td>?: Conditional</td></tr></table> The operator precedence for SystemVerilog is much like you would expect in other programming languages. In particular, AND has precedence over OR. We could take advantage of this precedence to eliminate the parentheses: assign cout = g p & cin;	Op	Meaning	H i g h e s t	~ NOT		*, /, % MUL, DIV, MOD		+, - PLUS, MINUS		<<, >> Logical Left/Right Shift		<<<, >>> Arithmetic Left/Right Shift		<, <=, >, >= Relative Comparison		==, != Equality Comparison	L o w e s t	&, ~& AND, NAND		^, ~^ XOR, XNOR		, ~ OR, NOR		?: Conditional	Table 4.2 VHDL operator precedence <table><tr><th>Op</th><th>Meaning</th></tr><tr><td>H i g h e s t</td><td>not NOT</td></tr><tr><td></td><td>*, /, mod, rem MUL, DIV, MOD, REM</td></tr><tr><td></td><td>+, - PLUS, MINUS</td></tr><tr><td></td><td>rol, ror, srl, sll Rotate, Shift logical</td></tr><tr><td>L o w e s t</td><td><, <=, >, >= Relative Comparison</td></tr><tr><td></td><td>=, /= Equality Comparison</td></tr><tr><td></td><td>and, or, nand, nor, xor, xnor Logical Operations</td></tr></table> Multiplication has precedence over addition in VHDL, as you would expect. However, unlike SystemVerilog, all of the logical operations (and, or, etc.) have equal precedence, unlike what one might expect in Boolean algebra. Thus, parentheses are necessary. Otherwise, cout <= g or p and cin would be interpreted from left to right as cout <= (g or p) and cin.	Op	Meaning	H i g h e s t	not NOT		*, /, mod, rem MUL, DIV, MOD, REM		+, - PLUS, MINUS		rol, ror, srl, sll Rotate, Shift logical	L o w e s t	<, <=, >, >= Relative Comparison		=, /= Equality Comparison		and, or, nand, nor, xor, xnor Logical Operations
Op	Meaning																																								
H i g h e s t	~ NOT																																								
	*, /, % MUL, DIV, MOD																																								
	+, - PLUS, MINUS																																								
	<<, >> Logical Left/Right Shift																																								
	<<<, >>> Arithmetic Left/Right Shift																																								
	<, <=, >, >= Relative Comparison																																								
	==, != Equality Comparison																																								
L o w e s t	&, ~& AND, NAND																																								
	^, ~^ XOR, XNOR																																								
	, ~ OR, NOR																																								
	?: Conditional																																								
Op	Meaning																																								
H i g h e s t	not NOT																																								
	*, /, mod, rem MUL, DIV, MOD, REM																																								
	+, - PLUS, MINUS																																								
	rol, ror, srl, sll Rotate, Shift logical																																								
L o w e s t	<, <=, >, >= Relative Comparison																																								
	=, /= Equality Comparison																																								
	and, or, nand, nor, xor, xnor Logical Operations																																								

specifies operator precedence from highest to lowest for each language. The tables include arithmetic, shift, and comparison operators that will be defined in [Chapter 5](#).

4.2.7 Numbers

Numbers can be specified in binary, octal, decimal, or hexadecimal (bases 2, 8, 10, and 16, respectively). The size, that is, the number of bits, may be given optionally, and leading zeros are inserted to reach this size. Underscores in numbers are ignored and can be helpful in breaking long numbers into more readable chunks. [HDL Example 4.9](#) explains how numbers are written in each language.

HDL Example 4.9 NUMBERS

SystemVerilog

The format for declaring constants is `N'Bvalue`, where `N` is the size in bits, `B` is a letter indicating the base, and `value` gives the value. For example, `9'h25` indicates a 9-bit number with a value of $25_{16} = 37_{10} = 000100101_2$. SystemVerilog supports `'b` for binary, `'o` for octal, `'d` for decimal, and `'h` for hexadecimal.

If the base is omitted, it defaults to decimal. If the size is not given, the number is assumed to have as many bits as the expression in which it is being used. Zeros are automatically padded on the front of the number to bring it up to full size. For example, if `w` is a 6-bit bus, `assign w = 'b11` gives `w` the value `000011`. It is better practice to explicitly give the size. An exception is that `'0` and `'1` are SystemVerilog idioms to fill all of the bits with 0 and 1, respectively.

Table 4.3 SystemVerilog numbers

Numbers	Bits	Base	Val	Stored
3'b101	3	2	5	101
'b11	?	2	3	000 ... 0011
8'b11	8	2	3	00000011
8'b1010_1011	8	2	171	10101011
3'd6	3	10	6	110
6'o42	6	8	34	100010
8'hAB	8	16	171	10101011
42	?	10	42	00 ... 0101010

VHDL

In VHDL, `STD_LOGIC` numbers are written in binary and enclosed in single quotes: `'0` and `'1` indicate logic 0 and 1. The format for declaring `STD_LOGIC_VECTOR` constants is `NB"value`", where `N` is the size in bits, `B` is a letter indicating the base, and `value` gives the value. For example, `9X"25`" indicates a 9-bit number with a value of $25_{16} = 37_{10} = 000100101_2$. VHDL 2008 supports `B` for binary, `O` for octal, `D` for decimal, and `X` for hexadecimal.

If the base is omitted, it defaults to binary. If the size is not given, the number is assumed to have a size matching the number of bits specified in the value. For example, `y <= X"7B"` requires that `y` be an 8-bit signal. If `y` has more bits, VHDL does *not* pad the number with 0's on the left. Instead, an error occurs during compilation. `others => '0` and `others => '1` are VHDL idioms to fill all of the bits with 0 and 1, respectively.

Table 4.4 VHDL numbers

Numbers	Bits	Base	Val	Stored
3B"101"	3	2	5	101
B"11"	2	2	3	11
8B"11"	8	2	3	00000011
8B"1010_1011"	8	2	171	10101011
3D"6"	3	10	6	110
6O"42"	6	8	34	100010
8X"AB"	8	16	171	10101011
"101"	3	2	5	101
B"101"	3	2	5	101
X"AB"	8	16	171	10101011

4.2.8 Z's and X's

HDLs use `z` to indicate a floating value. `z` is particularly useful for describing a tristate buffer, whose output floats when the enable is 0. Recall from [Section 2.6.2](#) that a bus can be driven by several tristate buffers, exactly one of which should be enabled. [HDL Example 4.10](#) shows the idiom for a tristate buffer. If the buffer is enabled, the output is the same as the input. If the buffer is disabled, the output is assigned a floating value (`z`).

Similarly, HDLs use `x` to indicate an invalid logic level. If a bus is simultaneously driven to 0 and 1 by two enabled tristate buffers (or other gates), the result is `x`, indicating contention. If all of the tristate buffers driving a bus are simultaneously OFF, the bus will float, indicated by `z`.

At the start of simulation, state nodes such as flip-flop outputs are initialized to an unknown state (x in SystemVerilog and u in VHDL). This is helpful to track errors caused by forgetting to reset a flip-flop before its output is used.

HDL Example 4.10 TRISTATE BUFFER

SystemVerilog

```
module tristate(input logic [3:0] a,
               input logic   en,
               output tri   [3:0] y);

    assign y = en ? a : 4'bz;
endmodule
```

Notice that `y` is declared as `tri` rather than `logic`. `logic` signals can only have a single driver. Tristate busses can have multiple drivers, so they should be declared as a *net*. Two types of nets in SystemVerilog are called `tri` and `triereg`. Typically, exactly one driver on a net is active at a time, and the net takes on that value. If no driver is active, a `tri` floats (z), while a `triereg` retains the previous value. If no type is specified for an input or output, `tri` is assumed. Also, note that a `tri` output from a module can be used as a `logic` input to another module. [Section 4.7](#) further discusses nets with multiple drivers.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity tristate is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         en: in  STD_LOGIC;
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of tristate is
begin
    y <= a when en else "ZZZZ";
end;
```

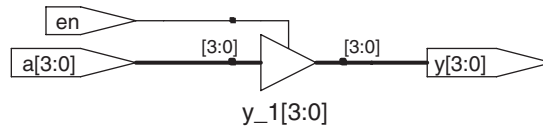


Figure 4.9 tristate synthesized circuit

If a gate receives a floating input, it may produce an x output when it can't determine the correct output value. Similarly, if it receives an illegal or uninitialized input, it may produce an x output. [HDL Example 4.11](#)

HDL Example 4.11 TRUTH TABLES WITH UNDEFINED AND FLOATING INPUTS

SystemVerilog

SystemVerilog signal values are 0, 1, z, and x. SystemVerilog constants starting with z or x are padded with leading z's or x's (instead of 0's) to reach their full length when necessary.

[Table 4.5](#) shows a truth table for an AND gate using all four possible signal values. Note that the gate can sometimes determine the output despite some inputs being unknown. For example, 0 & z returns 0 because the output of an AND gate is always 0 if either input is 0. Otherwise, floating or invalid inputs cause invalid outputs, displayed as x in SystemVerilog.

VHDL

VHDL STD_LOGIC signals are '0', '1', 'z', 'x', and 'u'.

[Table 4.6](#) shows a truth table for an AND gate using all five possible signal values. Notice that the gate can sometimes determine the output despite some inputs being unknown. For example, '0' and 'z' returns '0' because the output of an AND gate is always '0' if either input is '0'. Otherwise, floating or invalid inputs cause invalid outputs, displayed as 'x' in VHDL. Uninitialized inputs cause uninitialized outputs, displayed as 'u' in VHDL.

understand cause and effect (deducing the source of a bad output is tricky if all signals change simultaneously in the simulation results). These delays are ignored during synthesis; the delay of a gate produced by the synthesizer depends on its t_{pd} and t_{cd} specifications, not on numbers in HDL code.

HDL Example 4.13 adds delays to the original function from **HDL Example 4.1**, $y = \overline{a} \overline{b} \overline{c} + a \overline{b} \overline{c} + a \overline{b} c$. It assumes that inverters have a delay of 1 ns, three-input AND gates have a delay of 2 ns, and three-input OR gates have a delay of 4 ns. **Figure 4.10** shows the simulation waveforms, with y lagging 7 ns after the inputs. Note that y is initially unknown at the beginning of the simulation.

HDL Example 4.13 LOGIC GATES WITH DELAYS

SystemVerilog

```
'timescale 1ns/1ps

module example(input  logic a, b, c,
               output logic y);

    logic ab, bb, cb, n1, n2, n3;

    assign #1 {ab, bb, cb} = ~(a, b, c);
    assign #2 n1 = ab & bb & cb;
    assign #2 n2 = a & bb & cb;
    assign #2 n3 = a & bb & c;
    assign #4 y = n1 | n2 | n3;
endmodule
```

SystemVerilog files can include a timescale directive that indicates the value of each time unit. The statement is of the form `'timescale unit/precision`. In this file, each unit is 1 ns and the simulation has 1 ps precision. If no timescale directive is given in the file, a default unit and precision (usually 1 ns for both) are used. In SystemVerilog, a # symbol is used to indicate the number of units of delay. It can be placed in assign statements, as well as nonblocking (<=) and blocking (=) assignments, which will be discussed in [Section 4.5.4](#).

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity example is
    port(a, b, c: in  STD_LOGIC;
         y:      out STD_LOGIC);
end;

architecture synth of example is
    signal ab, bb, cb, n1, n2, n3: STD_LOGIC;
begin
    ab <= not a after 1 ns;
    bb <= not b after 1 ns;
    cb <= not c after 1 ns;
    n1 <= ab and bb and cb after 2 ns;
    n2 <= a and bb and cb after 2 ns;
    n3 <= a and bb and c after 2 ns;
    y  <= n1 or n2 or n3 after 4 ns;
end;
```

In VHDL, the `after` clause is used to indicate delay. The units, in this case, are specified as nanoseconds.

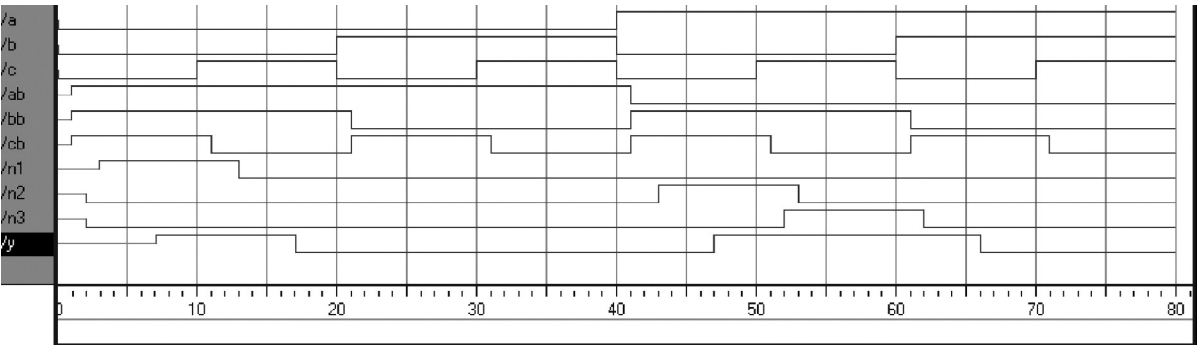


Figure 4.10 Example simulation waveforms with delays (from the ModelSim simulator)

4.3 STRUCTURAL MODELING

The previous section discussed *behavioral* modeling, describing a module in terms of the relationships between inputs and outputs. This section examines *structural* modeling, which describes a module in terms of how it is composed of simpler modules.

For example, [HDL Example 4.14](#) shows how to assemble a 4:1 multiplexer from three 2:1 multiplexers. Each copy of the 2:1 multiplexer is called

HDL Example 4.14 STRUCTURAL MODEL OF 4:1 MULTIPLEXER

SystemVerilog

```
module mux4(input  logic [3:0] d0, d1, d2, d3,
           input  logic [1:0] s,
           output logic [3:0] y);

    logic [3:0] low, high;

    mux2 lowmux(d0, d1, s[0], low);
    mux2 highmux(d2, d3, s[0], high);
    mux2 finalmux(low, high, s[1], y);
endmodule
```

The three mux2 instances are called lowmux, highmux, and finalmux. The mux2 module must be defined elsewhere in the SystemVerilog code—see [HDL Example 4.5](#), [4.15](#), or [4.34](#).

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4 is
    port(d0, d1,
         d2, d3: in  STD_LOGIC_VECTOR(3 downto 0);
         s:   in  STD_LOGIC_VECTOR(1 downto 0);
         y:   out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture struct of mux4 is
    component mux2
        port(d0,
             d1: in  STD_LOGIC_VECTOR(3 downto 0);
             s: in  STD_LOGIC;
             y: out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    signal low, high: STD_LOGIC_VECTOR(3 downto 0);
begin
    lowmux:  mux2 port map(d0, d1, s(0), low);
    highmux: mux2 port map(d2, d3, s(0), high);
    finalmux: mux2 port map(low, high, s(1), y);
end;
```

The architecture must first declare the mux2 ports using the component declaration statement. This allows VHDL tools to check that the component you wish to use has the same ports as the entity that was declared somewhere else in another entity statement, preventing errors caused by changing the entity but not the instance. However, component declaration makes VHDL code rather cumbersome.

Note that this architecture of mux4 was named `struct`, whereas architectures of modules with behavioral descriptions from [Section 4.2](#) were named `synth`. VHDL allows multiple architectures (implementations) for the same entity; the architectures are distinguished by name. The names themselves have no significance to the CAD tools, but `struct` and `synth` are common. Synthesizable VHDL code generally contains only one architecture for each entity, so we will not discuss the VHDL syntax to configure which architecture is used when multiple architectures are defined.

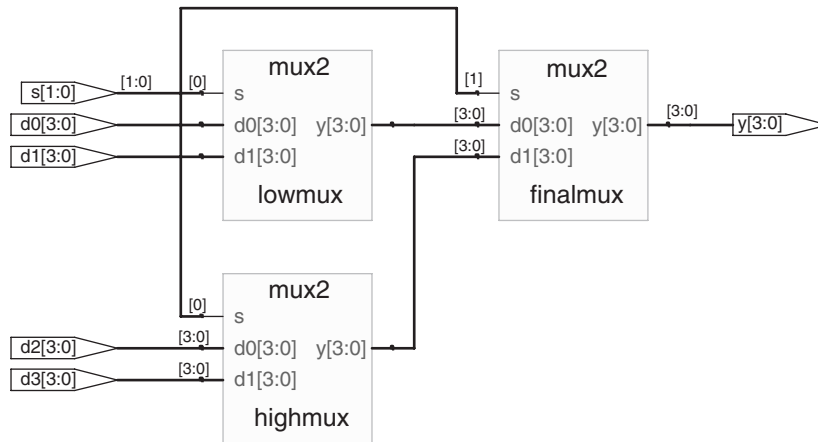


Figure 4.11 mux4 synthesized circuit

an *instance*. Multiple instances of the same module are distinguished by distinct names, in this case `lowmux`, `highmux`, and `finalmux`. This is an example of regularity, in which the 2:1 multiplexer is reused many times.

HDL Example 4.15 uses structural modeling to construct a 2:1 multiplexer from a pair of tristate buffers. Building logic out of tristates is not recommended, however.

HDL Example 4.15 STRUCTURAL MODEL OF 2:1 MULTIPLEXER

SystemVerilog

```

module mux2(input logic [3:0] d0, d1,
            input logic      s,
            output tri  [3:0] y);

    tristate t0(d0, ~s, y);
    tristate t1(d1, s, y);
endmodule

```

In SystemVerilog, expressions such as `~s` are permitted in the port list for an instance. Arbitrarily complicated expressions are legal but discouraged because they make the code difficult to read.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
    port(d0, d1: in  STD_LOGIC_VECTOR(3 downto 0);
          s:      in  STD_LOGIC;
          y:      out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture struct of mux2 is
    component tristate
        port(a: in  STD_LOGIC_VECTOR(3 downto 0);
              en: in  STD_LOGIC;
              y: out STD_LOGIC_VECTOR(3 downto 0));
    end component;
    signal sbar: STD_LOGIC;
begin
    sbar <= not s;
    t0: tristate port map(d0, sbar, y);
    t1: tristate port map(d1, s, y);
end;

```

In VHDL, expressions such as `not s` are not permitted in the port map for an instance. Thus, `sbar` must be defined as a separate signal.

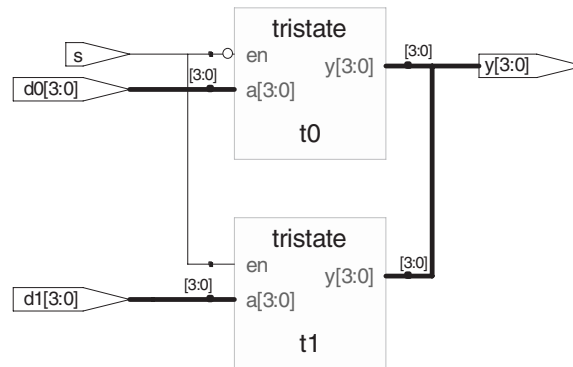


Figure 4.12 mux2 synthesized circuit

HDL Example 4.16 shows how modules can access part of a bus. An 8-bit-wide 2:1 multiplexer is built using two of the 4-bit 2:1 multiplexers already defined, operating on the low and high nibbles of the byte.

In general, complex systems are designed *hierarchically*. The overall system is described structurally by instantiating its major components. Each of these components is described structurally from its building blocks and so forth recursively until the pieces are simple enough to describe behaviorally. It is good style to avoid (or at least to minimize) mixing structural and behavioral descriptions within a single module.

HDL Example 4.16 ACCESSING PARTS OF BUSSES

SystemVerilog

```
module mux2_8(input  logic [7:0] d0, d1,
             input  logic    s,
             output logic [7:0] y);

    mux2 lsbmux (d0[3:0], d1[3:0], s, y[3:0]);
    mux2 msbmux (d0[7:4], d1[7:4], s, y[7:4]);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2_8 is
    port(d0, d1: in  STD_LOGIC_VECTOR(7 downto 0);
         s:    in  STD_LOGIC;
         y:    out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture struct of mux2_8 is
    component mux2
        port(d0, d1: in  STD_LOGIC_VECTOR(3 downto 0);
             s:    in  STD_LOGIC;
             y:    out STD_LOGIC_VECTOR(3 downto 0));
    end component;
begin

    lsbmux: mux2
        port map(d0(3 downto 0), d1(3 downto 0),
                 s, y(3 downto 0));

    msbmux: mux2
        port map(d0(7 downto 4), d1(7 downto 4),
                 s, y(7 downto 4));

end;
```

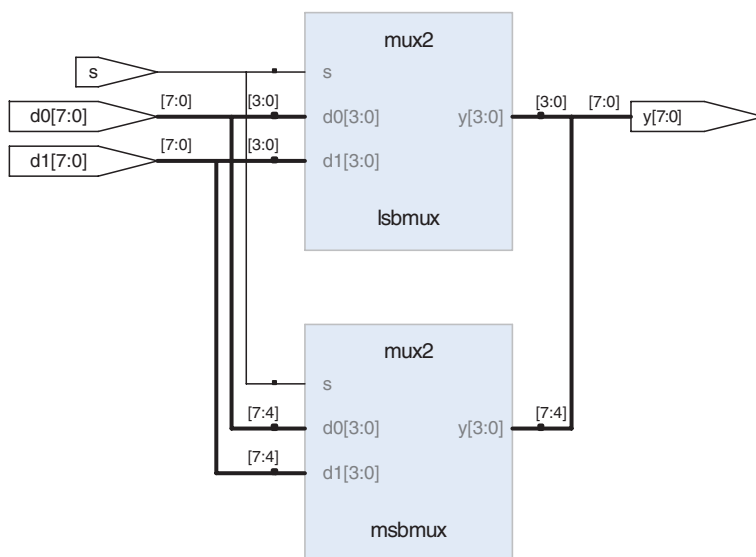


Figure 4.13 mux2_8 synthesized circuit

4.4 SEQUENTIAL LOGIC

HDL synthesizers recognize certain idioms and turn them into specific sequential circuits. Other coding styles may simulate correctly but synthesize into circuits with blatant or subtle errors. This section presents the proper idioms to describe registers and latches.

4.4.1 Registers

The vast majority of modern commercial systems are built with registers using positive edge-triggered D flip-flops. [HDL Example 4.17](#) shows the idiom for such flip-flops.

In SystemVerilog `always` statements and VHDL process statements, signals keep their old value until an event in the sensitivity list takes place that explicitly causes them to change. Hence, such code, with appropriate sensitivity lists, can be used to describe sequential circuits with memory. For example, the flip-flop includes only `clk` in the sensitive list. It remembers its old value of `q` until the next rising edge of the `clk`, even if `d` changes in the interim.

In contrast, SystemVerilog continuous assignment statements (`assign`) and VHDL concurrent assignment statements (`<=`) are reevaluated whenever any of the inputs on the right-hand side change. Therefore, such code necessarily describes combinational logic.

HDL Example 4.17 REGISTER**SystemVerilog**

```

module flop(input logic      clk,
            input logic [3:0] d,
            output logic [3:0] q);

    always_ff @(posedge clk)
        q <= d;
endmodule

```

In general, a SystemVerilog `always` statement is written in the form

```

always @(sensitivity list)
    statement;

```

The statement is executed only when the event specified in the `sensitivity list` occurs. In this example, the statement is `q <= d` (pronounced “q gets d”). Hence, the flip-flop copies `d` to `q` on the positive edge of the clock and otherwise remembers the old state of `q`. Note that sensitivity lists are also referred to as stimulus lists.

`<=` is called a *nonblocking assignment*. Think of it as a regular `=` sign for now; we’ll return to the more subtle points in [Section 4.5.4](#). Note that `<=` is used instead of `assign` inside an `always` statement.

As will be seen in subsequent sections, `always` statements can be used to imply flip-flops, latches, or combinational logic, depending on the sensitivity list and statement. Because of this flexibility, it is easy to produce the wrong hardware inadvertently. SystemVerilog introduces `always_ff`, `always_latch`, and `always_comb` to reduce the risk of common errors. `always_ff` behaves like `always` but is used exclusively to imply flip-flops and allows tools to produce a warning if anything else is implied.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flop is
    port(clk: in  STD_LOGIC;
         d:  in  STD_LOGIC_VECTOR(3 downto 0);
         q:  out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of flop is
begin
    process(clk) begin
        if rising_edge(clk) then
            q <= d;
        end if;
    end process;
end;

```

A VHDL process is written in the form

```

process(sensitivity list) begin
    statement;
end process;

```

The statement is executed when any of the variables in the `sensitivity list` change. In this example, the `if` statement checks whether the change was a rising edge on `clk`. If so, then `q <= d` (pronounced “q gets d”). Hence, the flip-flop copies `d` to `q` on the positive edge of the clock and otherwise remembers the old state of `q`.

An alternative VHDL idiom for a flip-flop is

```

process(clk) begin
    if clk'event and clk = '1' then
        q <= d;
    end if;
end process;

```

`rising_edge(clk)` is synonymous with `clk'event and clk = '1'`.

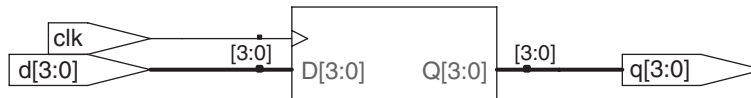


Figure 4.14 flop synthesized circuit

4.4.2 Resettable Registers

When simulation begins or power is first applied to a circuit, the output of a flop or register is unknown. This is indicated with `x` in SystemVerilog and `u` in VHDL. Generally, it is good practice to use resettable registers so that on powerup you can put your system in a known state. The reset may be either asynchronous or synchronous. Recall that asynchronous reset occurs immediately, whereas synchronous reset clears the output only on

the next rising edge of the clock. [HDL Example 4.18](#) demonstrates the idioms for flip-flops with asynchronous and synchronous resets. Note that distinguishing synchronous and asynchronous reset in a schematic can be difficult.

HDL Example 4.18 RESETTABLE REGISTER

SystemVerilog

```
module flopr(input logic clk,
            input logic reset,
            input logic [3:0] d,
            output logic [3:0] q);

    // asynchronous reset
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule

module flopr(input logic clk,
            input logic reset,
            input logic [3:0] d,
            output logic [3:0] q);

    // synchronous reset
    always_ff @(posedge clk)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule
```

Multiple signals in an always statement sensitivity list are separated with a comma or the word `or`. Notice that `posedge reset` is in the sensitivity list on the asynchronously resettable flop, but not on the synchronously resettable flop. Thus, the asynchronously resettable flop immediately responds to a rising edge on `reset`, but the synchronously resettable flop responds to `reset` only on the rising edge of the clock.

Because the modules have the same name, `flopr`, you may include only one or the other in your design. If you wanted to use both, you would need to rename one of the modules.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flopr is
    port(clk, reset: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(3 downto 0);
         q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture asynchronous of flopr is
begin
    process(clk, reset) begin
        if reset then
            q <= "0000";
        elsif rising_edge(clk) then
            q <= d;
        end if;
    end process;
end;

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flopr is
    port(clk, reset: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(3 downto 0);
         q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synchronous of flopr is
begin
    process(clk) begin
        if rising_edge(clk) then
            if reset then q <= "0000";
            else q <= d;
            end if;
        end if;
    end process;
end;
```

Multiple signals in a process sensitivity list are separated with a comma. Notice that `reset` is in the sensitivity list on the asynchronously resettable flop but not on the synchronously resettable flop. Thus, the asynchronously resettable flop immediately responds to a rising edge on `reset`, but the synchronously resettable flop responds to `reset` only on the rising edge of the clock.

As mentioned earlier, the name of the architecture (asynchronous or synchronous, in this example) is ignored by the VHDL tools but may be helpful to the human reading the code. Because both architectures describe the entity `flopr`, you may include only one or the other in your design. If you wanted to use both, you would need to rename one of the modules.

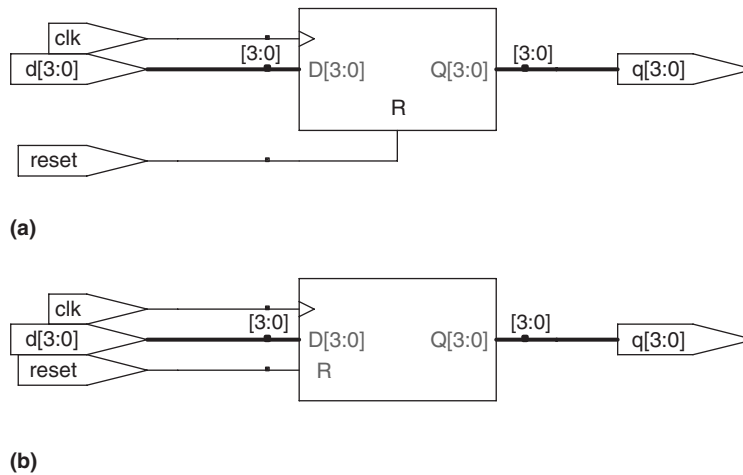


Figure 4.15 flopr synthesized circuit (a) asynchronous reset, (b) synchronous reset

4.4.3 Enabled Registers

Enabled registers respond to the clock only when the enable is asserted. [HDL Example 4.19](#) shows an asynchronously resettable enabled register that retains its old value if both `reset` and `en` are `FALSE`.

HDL Example 4.19 RESETTABLE ENABLED REGISTER

SystemVerilog

```
module flopenr(input logic    clk,
               input logic    reset,
               input logic    en,
               input logic [3:0] d,
               output logic [3:0] q);

    // asynchronous reset
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else if (en) q <= d;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity flopenr is
    port(clk,
          reset,
          en: in STD_LOGIC;
          d: in STD_LOGIC_VECTOR(3 downto 0);
          q: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture asynchronous of flopenr is
    -- asynchronous reset
    begin
        process(clk, reset) begin
            if reset then
                q <= "0000";
            elsif rising_edge(clk) then
                if en then
                    q <= d;
                end if;
            end if;
        end process;
    end;
end;
```

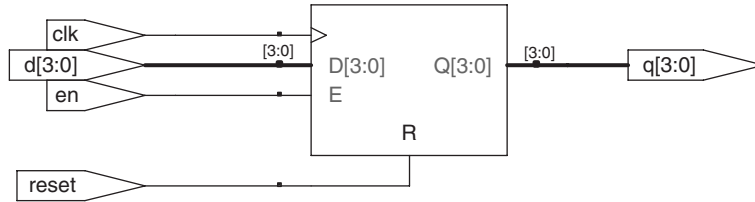



Figure 4.16 flopenr synthesized circuit

4.4.4 Multiple Registers

A single always/process statement can be used to describe multiple pieces of hardware. For example, consider the synchronizer from Section 3.5.5 made of two back-to-back flip-flops, as shown in Figure 4.17. HDL Example 4.20 describes the synchronizer. On the rising edge of *clk*, *d* is copied to *n1*. At the same time, *n1* is copied to *q*.

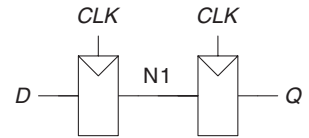


Figure 4.17 Synchronizer circuit

HDL Example 4.20 SYNCHRONIZER

SystemVerilog

```
module sync(input logic clk,
            input logic d,
            output logic q);

    logic n1;

    always_ff @(posedge clk)
    begin
        n1 <= d; // nonblocking
        q <= n1; // nonblocking
    end
endmodule
```

Notice that the begin/end construct is necessary because multiple statements appear in the always statement. This is analogous to {} in C or Java. The begin/end was not needed in the flopr example because if/else counts as a single statement.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity sync is
    port(clk: in STD_LOGIC;
          d: in STD_LOGIC;
          q: out STD_LOGIC);
end;

architecture good of sync is
    signal n1: STD_LOGIC;
begin
    process(clk) begin
        if rising_edge(clk) then
            n1 <= d;
            q <= n1;
        end if;
    end process;
end;
```

n1 must be declared as a signal because it is an internal signal used in the module.

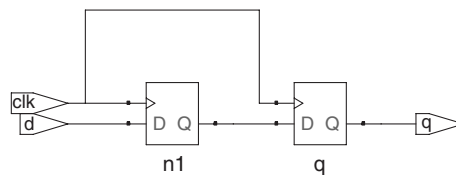


Figure 4.18 sync synthesized circuit

4.4.5 Latches

Recall from [Section 3.2.2](#) that a D latch is transparent when the clock is HIGH, allowing data to flow from input to output. The latch becomes opaque when the clock is LOW, retaining its old state. [HDL Example 4.21](#) shows the idiom for a D latch.

Not all synthesis tools support latches well. Unless you know that your tool does support latches and you have a good reason to use them, avoid them and use edge-triggered flip-flops instead. Furthermore, take care that your HDL does not imply any unintended latches, something that is easy to do if you aren't attentive. Many synthesis tools warn you when a latch is created; if you didn't expect one, track down the bug in your HDL. And if you don't know whether you intended to have a latch or not, you are probably approaching HDLs like a programming language and have bigger problems lurking.

4.5 MORE COMBINATIONAL LOGIC

In [Section 4.2](#), we used assignment statements to describe combinational logic behaviorally. SystemVerilog `always` statements and VHDL `process`

HDL Example 4.21 D LATCH

SystemVerilog

```
module latch(input logic      clk,
             input logic [3:0] d,
             output logic [3:0] q);

    always_latch
        if (clk) q <= d;
endmodule
```

`always_latch` is equivalent to `always @(clk, d)` and is the preferred idiom for describing a latch in SystemVerilog. It evaluates whenever `clk` or `d` changes. If `clk` is HIGH, `d` flows through to `q`, so this code describes a positive level-sensitive latch. Otherwise, `q` keeps its old value. SystemVerilog can generate a warning if the `always_latch` block doesn't imply a latch.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity latch is
    port(clk: in  STD_LOGIC;
         d:  in  STD_LOGIC_VECTOR(3 downto 0);
         q:  out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of latch is
begin
    process(clk, d) begin
        if clk = '1' then
            q <= d;
        end if;
    end process;
end;
```

The sensitivity list contains both `clk` and `d`, so the process evaluates whenever `clk` or `d` changes. If `clk` is HIGH, `d` flows through to `q`.



Figure 4.19 latch synthesized circuit

HDL Example 4.22 INVERTER USING `always/process`**SystemVerilog**

```
module inv(input logic [3:0] a,
          output logic [3:0] y);

    always_comb
        y = ~a;
endmodule
```

`always_comb` reevaluates the statements inside the `always` statement whenever any of the signals on the right-hand side of `<=` or `=` in the `always` statement change. In this case, it is equivalent to `always @(a)` but is better because it avoids mistakes if signals in the `always` statement are renamed or added. If the code inside the `always` block is not combinational logic, SystemVerilog will report a warning. `always_comb` is equivalent to `always @(*)` but is preferred in SystemVerilog.

The `=` in the `always` statement is called a *blocking assignment*, in contrast to the `<=` nonblocking assignment. In SystemVerilog, it is good practice to use blocking assignments for combinational logic and nonblocking assignments for sequential logic. This will be discussed further in [Section 4.5.4](#).

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity inv is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
          y: out STD_LOGIC_VECTOR(3 downto 0));
end;
```

```
architecture proc of inv is
begin
    process(all) begin
        y <= not a;
    end process;
end;
```

`process(all)` reevaluates the statements inside the process whenever any of the signals in the process change. It is equivalent to `process(a)` but is better because it avoids mistakes if signals in the process are renamed or added.

The `begin` and `end` process statements are required in VHDL even though the process contains only one assignment.

statements are used to describe sequential circuits because they remember the old state when no new state is prescribed. However, `always/process` statements can also be used to describe combinational logic behaviorally if the sensitivity list is written to respond to changes in all of the inputs and the body prescribes the output value for every possible input combination. [HDL Example 4.22](#) uses `always/process` statements to describe a bank of four inverters (see [Figure 4.3](#) for the synthesized circuit).

HDLs support *blocking* and *nonblocking assignments* in an `always/process` statement. A group of blocking assignments are evaluated in the order in which they appear in the code, just as one would expect in a standard programming language. A group of nonblocking assignments are evaluated concurrently; all of the statements are evaluated before any of the signals on the left-hand sides are updated.

[HDL Example 4.23](#) defines a full adder using intermediate signals `p` and `g` to compute `s` and `cout`. It produces the same circuit from [Figure 4.8](#), but uses `always/process` statements in place of assignment statements.

[HDL Examples 4.22](#) and [4.23](#) are poor applications of `always/process` statements for modeling combinational logic because they require more lines than the equivalent approach with assignment statements from [HDL Examples 4.2](#) and [4.7](#). However, `case` and `if` statements are convenient for modeling more complicated combinational logic. `case` and `if` statements must appear within `always/process` statements and are examined in the next sections.

SystemVerilog

In a SystemVerilog `always` statement, `=` indicates a blocking assignment and `<=` indicates a nonblocking assignment (also called a concurrent assignment).

Do not confuse either type with continuous assignment, i.e., the `assign` statement. `assign` statements must be used outside `always` statements and are also evaluated concurrently.

VHDL

In a VHDL process statement, `:=` indicates a blocking assignment and `<=` indicates a nonblocking assignment (also called a concurrent assignment). This is the first section where `:=` is introduced.

Nonblocking assignments are made to outputs and signals. Blocking assignments are made to variables, which are declared in process statements (see [HDL Example 4.23](#)). `<=` can also appear outside process statements, where it is also evaluated concurrently.

HDL Example 4.23 FULL ADDER USING `always/process`

SystemVerilog

```
module fulladder(input logic a, b, cin,
                 output logic s, cout);

    logic p, g;

    always_comb
    begin
        p = a ^ b;           // blocking
        g = a & b;           // blocking
        s = p ^ cin;         // blocking
        cout = g | (p & cin); // blocking
    end
endmodule
```

In this case, `always @(a, b, cin)` would have been equivalent to `always_comb`. However, `always_comb` is better because it avoids common mistakes of missing signals in the sensitivity list.

For reasons that will be discussed in [Section 4.5.4](#), it is best to use blocking assignments for combinational logic. This example uses blocking assignments, first computing `p`, then `g`, then `s`, and, finally, `cout`.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity fulladder is
    port(a, b, cin: in  STD_LOGIC;
         s, cout: out STD_LOGIC);
end;

architecture synth of fulladder is
begin
    process(all)
        variable p, g: STD_LOGIC;
    begin
        p := a xor b; -- blocking
        g := a and b; -- blocking
        s <= p xor cin;
        cout <= g or (p and cin);
    end process;
end;
```

In this case, `process(a, b, cin)` would have been equivalent to `process(all)`. However, `process(all)` is better because it avoids common mistakes of missing signals in the sensitivity list.

For reasons that will be discussed in [Section 4.5.4](#), it is best to use blocking assignments for intermediate variables in combinational logic. This example uses blocking assignments for `p` and `g` so that they get their new values before being used to compute `s` and `cout` that depend on them.

Because `p` and `g` appear on the left-hand side of a blocking assignment (`:=`) in a process statement, they must be declared to be variable rather than signal. The variable declaration appears before the `begin` in the process where the variable is used.

4.5.1 Case Statements

A better application of using the `always/process` statement for combinational logic is a seven-segment display decoder that takes advantage of the case statement that must appear inside an `always/process` statement.

As you might have noticed in the seven-segment display decoder of Example 2.10, the design process for large blocks of combinational logic is tedious and prone to error. HDLs offer a great improvement, allowing you to specify the function at a higher level of abstraction and then automatically synthesize the function into gates. [HDL Example 4.24](#) uses case statements to describe a seven-segment display decoder based on its truth table. The case statement performs different actions depending on the value of its input. A case statement implies combinational logic if all

HDL Example 4.24 SEVEN-SEGMENT DISPLAY DECODER

SystemVerilog

```
module sevenseg(input logic [3:0] data,
               output logic [6:0] segments);
  always_comb
  case(data)
    //                abc_defg
    0: segments = 7'b111_1110;
    1: segments = 7'b011_0000;
    2: segments = 7'b110_1101;
    3: segments = 7'b111_1001;
    4: segments = 7'b011_0011;
    5: segments = 7'b101_1011;
    6: segments = 7'b101_1111;
    7: segments = 7'b111_0000;
    8: segments = 7'b111_1111;
    9: segments = 7'b111_0011;
    default: segments = 7'b000_0000;
  endcase
endmodule
```

The case statement checks the value of `data`. When `data` is 0, the statement performs the action after the colon, setting `segments` to 1111110. The case statement similarly checks other data values up to 9 (note the use of the default base, base 10).

The default clause is a convenient way to define the output for all cases not explicitly listed, guaranteeing combinational logic.

In SystemVerilog, case statements must appear inside `always` statements.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity seven_seg_decoder is
  port(data: in STD_LOGIC_VECTOR(3 downto 0);
        segments: out STD_LOGIC_VECTOR(6 downto 0));
end;

architecture synth of seven_seg_decoder is
begin
  process(all) begin
    case data is
      --                abcdefg
      when X"0" => segments <= "1111110";
      when X"1" => segments <= "0110000";
      when X"2" => segments <= "1101101";
      when X"3" => segments <= "1111001";
      when X"4" => segments <= "0110011";
      when X"5" => segments <= "1011011";
      when X"6" => segments <= "1011111";
      when X"7" => segments <= "1110000";
      when X"8" => segments <= "1111111";
      when X"9" => segments <= "1110011";
      when others => segments <= "0000000";
    end case;
  end process;
end;
```

The case statement checks the value of `data`. When `data` is 0, the statement performs the action after the `=>`, setting `segments` to 1111110. The case statement similarly checks other data values up to 9 (note the use of `X` for hexadecimal numbers). The `others` clause is a convenient way to define the output for all cases not explicitly listed, guaranteeing combinational logic.

Unlike SystemVerilog, VHDL supports selected signal assignment statements (see [HDL Example 4.6](#)), which are much like case statements but can appear outside processes. Thus, there is less reason to use processes to describe combinational logic.

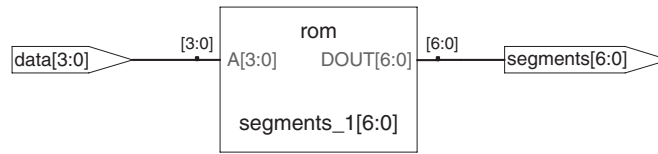


Figure 4.20 sevenseg synthesized circuit

possible input combinations are defined. Otherwise, it implies sequential logic, because the output will keep its old value in the undefined cases.

The HDL for the seven-segment display decoder synthesizes into a *read-only memory* (ROM) containing the 7 outputs for each of the 16 possible inputs. ROMs are discussed further in [Section 5.5.6](#).

If the `default` or `others` clause were left out of the case statement, the decoder would have remembered its previous output any time data were in the range of 10–15. This is strange behavior for hardware.

Ordinary decoders are also commonly written with case statements. [HDL Example 4.25](#) describes a 3:8 decoder.

4.5.2 If Statements

`always/process` statements may also contain if statements. The if statement may be followed by an else statement. If all possible input

HDL Example 4.25 3:8 DECODER

SystemVerilog

```
module decoder3_8(input logic [2:0] a,
                 output logic [7:0] y);

    always_comb
    case(a)
        3'b000: y=8'b00000001;
        3'b001: y=8'b00000010;
        3'b010: y=8'b00000100;
        3'b011: y=8'b00001000;
        3'b100: y=8'b00010000;
        3'b101: y=8'b00100000;
        3'b110: y=8'b01000000;
        3'b111: y=8'b10000000;
        default: y=8'bxxxxxxxx;
    endcase
endmodule
```

The `default` statement isn't strictly necessary for logic synthesis in this case because all possible input combinations are defined, but it is prudent for simulation in case one of the inputs is an x or z.

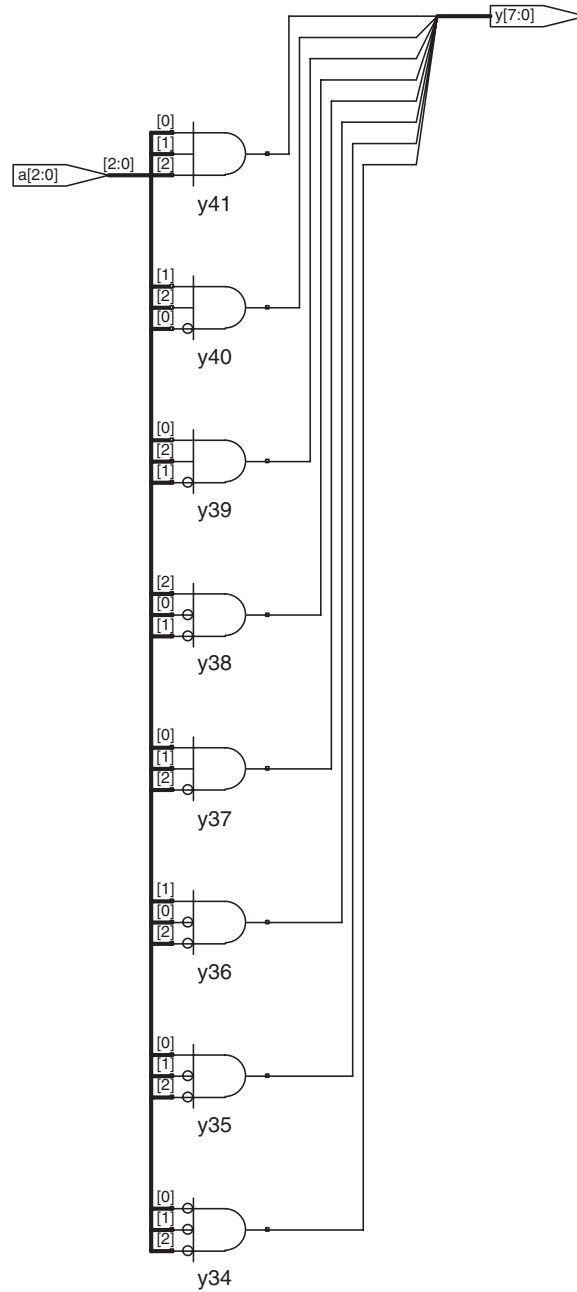
VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity decoder3_8 is
    port(a: in STD_LOGIC_VECTOR(2 downto 0);
         y: out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture synth of decoder3_8 is
begin
    process(all) begin
        case a is
            when "000" => y <= "00000001";
            when "001" => y <= "00000010";
            when "010" => y <= "00000100";
            when "011" => y <= "00001000";
            when "100" => y <= "00010000";
            when "101" => y <= "00100000";
            when "110" => y <= "01000000";
            when "111" => y <= "10000000";
            when others => y <= "XXXXXXXX";
        end case;
    end process;
end;
```

The `others` clause isn't strictly necessary for logic synthesis in this case because all possible input combinations are defined, but it is prudent for simulation in case one of the inputs is an x, z, or u.

**Figure 4.21** decoder3_8 synthesized circuit

HDL Example 4.26 PRIORITY CIRCUIT**SystemVerilog**

```

module priorityckt(input logic [3:0] a,
                  output logic [3:0] y);
    always_comb
    if (a[3]) y = 4'b1000;
    else if (a[2]) y = 4'b0100;
    else if (a[1]) y = 4'b0010;
    else if (a[0]) y = 4'b0001;
    else
        y = 4'b0000;
endmodule

```

In SystemVerilog, if statements must appear inside of always statements.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity priorityckt is
    port(a: in STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture synth of priorityckt is
begin
    process(all) begin
        if a(3) then y <= "1000";
        elsif a(2) then y <= "0100";
        elsif a(1) then y <= "0010";
        elsif a(0) then y <= "0001";
        else
            y <= "0000";
        end if;
    end process;
end;

```

Unlike SystemVerilog, VHDL supports conditional signal assignment statements (see [HDL Example 4.6](#)), which are much like if statements but can appear outside processes. Thus, there is less reason to use processes to describe combinational logic.

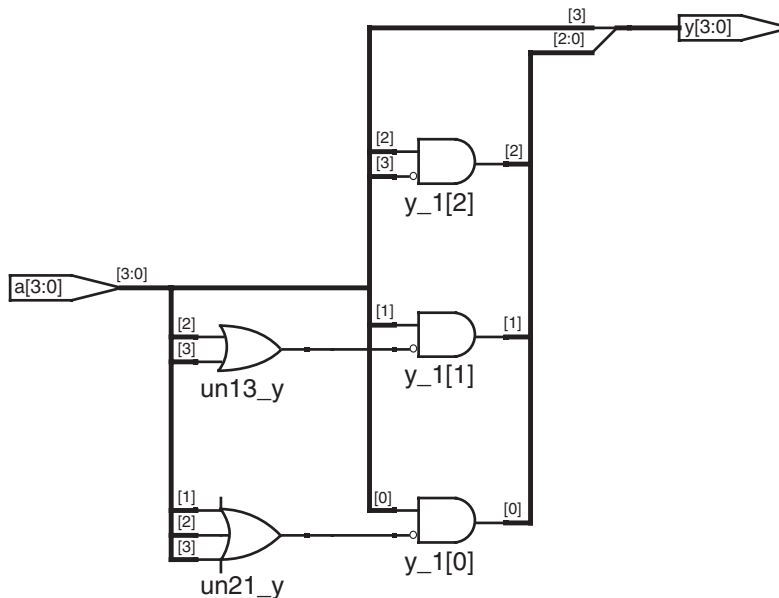


Figure 4.22 priorityckt synthesized circuit

combinations are handled, the statement implies combinational logic. Otherwise, it produces sequential logic (like the latch in [Section 4.4.5](#)).

[HDL Example 4.26](#) uses `if` statements to describe a priority circuit, defined in [Section 2.4](#). Recall that an N -input priority circuit sets the output TRUE that corresponds to the most significant input that is TRUE.

4.5.3 Truth Tables with Don't Cares

As examined in [Section 2.7.3](#), truth tables may include don't cares to allow more logic simplification. [HDL Example 4.27](#) shows how to describe a priority circuit with don't cares.

The synthesis tool generates a slightly different circuit for this module, shown in [Figure 4.23](#), than it did for the priority circuit in [Figure 4.22](#). However, the circuits are logically equivalent.

4.5.4 Blocking and Nonblocking Assignments

The guidelines on page 204 explain when and how to use each type of assignment. If these guidelines are not followed, it is possible to write code that appears to work in simulation but synthesizes to incorrect hardware. The optional remainder of this section explains the principles behind the guidelines.

HDL Example 4.27 PRIORITY CIRCUIT USING DON'T CARES

SystemVerilog

```
module priority_casez(input  logic [3:0] a,
                    output logic [3:0] y);

    always_comb
        casez(a)
            4'b1???: y = 4'b1000;
            4'b01??: y = 4'b0100;
            4'b001?: y = 4'b0010;
            4'b0001: y = 4'b0001;
            default: y = 4'b0000;
        endcase
endmodule
```

The `casez` statement acts like a case statement except that it also recognizes `?` as don't care.

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity priority_casez is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(3 downto 0));
end;

architecture dontcare of priority_casez is
begin
    process(all) begin
        case? a is
            when "1---" => y <= "1000";
            when "01--" => y <= "0100";
            when "001-" => y <= "0010";
            when "0001" => y <= "0001";
            when others => y <= "0000";
        end case?;
    end process;
end;
```

The `case?` statement acts like a case statement except that it also recognizes `-` as don't care.

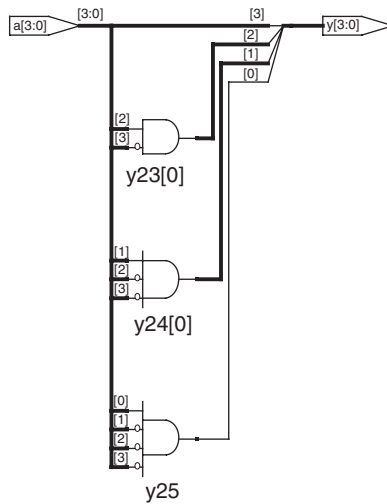


Figure 4.23 priority_casez synthesized circuit

BLOCKING AND NONBLOCKING ASSIGNMENT GUIDELINES

SystemVerilog

1. Use `always_ff @(posedge clk)` and nonblocking assignments to model synchronous sequential logic.

```
always_ff @(posedge clk)
begin
    n1 <= d; // nonblocking
    q <= n1; // nonblocking
end
```

2. Use continuous assignments to model simple combinational logic.

```
assign y = s ? d1 : d0;
```

3. Use `always_comb` and blocking assignments to model more complicated combinational logic where the `always` statement is helpful.

```
always_comb
if      (a[3]) y = 4'b1000;
else if (a[2]) y = 4'b0100;
else if (a[1]) y = 4'b0010;
else if (a[0]) y = 4'b0001;
else      y = 4'b0000;
```

4. Do not make assignments to the same signal in more than one `always` statement or continuous assignment statement.

VHDL

1. Use `process(clk)` and nonblocking assignments to model synchronous sequential logic.

```
process(clk) begin
    if rising_edge(clk) then
        n1 <= d; -- nonblocking
        q <= n1; -- nonblocking
    end if;
end process;
```

2. Use concurrent assignments outside `process` statements to model simple combinational logic.

```
y <= d0 when s = '0' else d1;
```

3. Use `process(all)` to model more complicated combinational logic where the `process` is helpful. Use blocking assignments for internal variables.

```
process(all)
    variable p, g: STD_LOGIC;
begin
    p := a xor b; -- blocking
    g := a and b; -- blocking
    s <= p xor cin;
    cout <= g or (p and cin);
end process;
```

4. Do not make assignments to the same variable in more than one `process` or concurrent assignment statement.

Combinational Logic*

The full adder from [HDL Example 4.23](#) is correctly modeled using blocking assignments. This section explores how it operates and how it would differ if nonblocking assignments had been used.

Imagine that *a*, *b*, and *cin* are all initially 0. *p*, *g*, *s*, and *cout* are thus 0 as well. At some time, *a* changes to 1, triggering the *always*/process statement. The four blocking assignments evaluate in the order shown here. (In the VHDL code, *s* and *cout* are assigned concurrently.) Note that *p* and *g* get their new values before *s* and *cout* are computed because of the blocking assignments. This is important because we want to compute *s* and *cout* using the new values of *p* and *g*.

1. $p \leftarrow 1 \oplus 0 = 1$
2. $g \leftarrow 1 \cdot 0 = 0$
3. $s \leftarrow 1 \oplus 0 = 1$
4. $\text{cout} \leftarrow 0 + 1 \cdot 0 = 0$

In contrast, [HDL Example 4.28](#) illustrates the use of nonblocking assignments.

Now, consider the same case of *a* rising from 0 to 1 while *b* and *cin* are 0. The four nonblocking assignments evaluate concurrently:

$$p \leftarrow 1 \oplus 0 = 1 \quad g \leftarrow 1 \cdot 0 = 0 \quad s \leftarrow 0 \oplus 0 = 0 \quad \text{cout} \leftarrow 0 + 0 \cdot 0 = 0$$

HDL Example 4.28 FULL ADDER USING NONBLOCKING ASSIGNMENTS**SystemVerilog**

```
// nonblocking assignments (not recommended)
module fulladder(input  logic a, b, cin,
                 output logic s, cout);

    logic p, g;

    always_comb
    begin
        p <= a ^ b; // nonblocking
        g <= a & b; // nonblocking

        s <= p ^ cin;
        cout <= g | (p & cin);
    end
endmodule
```

VHDL

```
-- nonblocking assignments (not recommended)
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity fulladder is
    port(a, b, cin: in  STD_LOGIC;
         s, cout:  out STD_LOGIC);
end;

architecture nonblocking of fulladder is
    signal p, g: STD_LOGIC;
begin
    process(all) begin
        p <= a xor b; -- nonblocking
        g <= a and b; -- nonblocking
        s <= p xor cin;
        cout <= g or (p and cin);
    end process;
end;
```

Because *p* and *g* appear on the left-hand side of a nonblocking assignment in a process statement, they must be declared to be signal rather than variable. The signal declaration appears before the begin in the architecture, not the process.

Observe that *s* is computed concurrently with *p*. Hence, it uses the old value of *p*, not the new value. Therefore, *s* remains 0 rather than becoming 1. However, *p* does change from 0 to 1. This change triggers the `always/process` statement to evaluate a second time, as follows:

$$p \leftarrow 1 \oplus 0 = 1 \quad g \leftarrow 1 \cdot 0 = 0 \quad s \leftarrow 1 \oplus 0 = 1 \quad \text{cout} \leftarrow 0 + 1 \cdot 0 = 0$$

This time, *p* is already 1, so *s* correctly changes to 1. The nonblocking assignments eventually reach the right answer, but the `always/process` statement had to evaluate twice. This makes simulation slower, though it synthesizes to the same hardware.

Another drawback of nonblocking assignments in modeling combinational logic is that the HDL will produce the wrong result if you forget to include the intermediate variables in the sensitivity list.

Worse yet, some synthesis tools will synthesize the correct hardware even when a faulty sensitivity list causes incorrect simulation. This leads to a mismatch between the simulation results and what the hardware actually does.

SystemVerilog	VHDL
If the sensitivity list of the <code>always</code> statement in HDL Example 4.28 were written as <code>always @(a, b, cin)</code> rather than <code>always_comb</code> , then the statement would not reevaluate when <i>p</i> or <i>g</i> changes. In that case, <i>s</i> would be incorrectly left at 0, not 1.	If the sensitivity list of the <code>process</code> statement in HDL Example 4.28 were written as <code>process(a, b, cin)</code> rather than <code>process(all)</code> , then the statement would not reevaluate when <i>p</i> or <i>g</i> changes. In that case, <i>s</i> would be incorrectly left at 0, not 1.

Sequential Logic*

The synchronizer from [HDL Example 4.20](#) is correctly modeled using nonblocking assignments. On the rising edge of the clock, *d* is copied to *n1* at the same time that *n1* is copied to *q*, so the code properly describes two registers. For example, suppose initially that *d* = 0, *n1* = 1, and *q* = 0. On the rising edge of the clock, the following two assignments occur concurrently, so that after the clock edge, *n1* = 0 and *q* = 1.

$$n1 \leftarrow d = 0 \quad q \leftarrow n1 = 1$$

[HDL Example 4.29](#) tries to describe the same module using blocking assignments. On the rising edge of *clk*, *d* is copied to *n1*. Then, this new value of *n1* is copied to *q*, resulting in *d* improperly appearing at both *n1* and *q*. The assignments occur one after the other so that after the clock edge, *q* = *n1* = 0.

1. *n1* ← *d* = 0
2. *q* ← *n1* = 0

HDL Example 4.29 BAD SYNCHRONIZER WITH BLOCKING ASSIGNMENTS**SystemVerilog**

```
// Bad implementation of a synchronizer using blocking
// assignments

module syncbad(input  logic clk,
               input  logic d,
               output logic q);

    logic n1;

    always_ff @(posedge clk)
    begin
        n1 = d; // blocking
        q = n1; // blocking
    end
endmodule
```

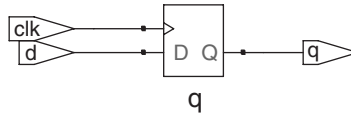
VHDL

```
-- Bad implementation of a synchronizer using blocking
-- assignment

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity syncbad is
    port(clk:in  STD_LOGIC;
         d: in  STD_LOGIC;
         q: out STD_LOGIC);
end;

architecture bad of syncbad is
begin
    process(clk)
        variable n1: STD_LOGIC;
    begin
        if rising_edge(clk) then
            n1 := d; -- blocking
            q <= n1;
        end if;
    end process;
end;
```

**Figure 4.24** synbad synthesized circuit

Because `n1` is invisible to the outside world and does not influence the behavior of `q`, the synthesizer optimizes it away entirely, as shown in Figure 4.24.

The moral of this illustration is to exclusively use nonblocking assignment in `always/process` statements when modeling sequential logic. With sufficient cleverness, such as reversing the orders of the assignments, you could make blocking assignments work correctly, but blocking assignments offer no advantages and only introduce the risk of unintended behavior. Certain sequential circuits will not work with blocking assignments no matter what the order.

4.6 FINITE STATE MACHINES

Recall that a finite state machine (FSM) consists of a state register and two blocks of combinational logic to compute the next state and the output given the current state and the input, as was shown in Figure 3.22. HDL descriptions of state machines are correspondingly divided into three parts to model the state register, the next state logic, and the output logic.

HDL Example 4.30 DIVIDE-BY-3 FINITE STATE MACHINE**SystemVerilog**

```

module divideby3FSM(input logic clk,
                   input logic reset,
                   output logic y);
    typedef enum logic [1:0] {S0, S1, S2} statetype;
    statetype state, nextstate;

    // state register
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else      state <= nextstate;

    // next state logic
    always_comb
        case (state)
            S0:      nextstate = S1;
            S1:      nextstate = S2;
            S2:      nextstate = S0;
            default: nextstate = S0;
        endcase

    // output logic
    assign y = (state == S0);
endmodule

```

The `typedef` statement defines `statetype` to be a two-bit logic value with three possibilities: S0, S1, or S2. `state` and `nextstate` are `statetype` signals.

The enumerated encodings default to numerical order: S0 = 00, S1 = 01, and S2 = 10. The encodings can be explicitly set by the user; however, the synthesis tool views them as suggestions, not requirements. For example, the following snippet encodes the states as 3-bit one-hot values:

```

typedef enum logic [2:0] {S0 = 3'b001, S1 = 3'b010, S2 = 3'b100}
statetype;

```

Notice how a `case` statement is used to define the state transition table. Because the next state logic should be combinational, a default is necessary, even though the state 2'b11 should never arise.

The output, `y`, is 1 when the state is S0. The *equality comparison* `a == b` evaluates to 1 if `a` equals `b` and 0 otherwise. The *inequality comparison* `a != b` does the inverse, evaluating to 1 if `a` does not equal `b`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity divideby3FSM is
    port(clk, reset: in  STD_LOGIC;
         y:      out STD_LOGIC);
end;

architecture synth of divideby3FSM is
    type statetype is (S0, S1, S2);
    signal state, nextstate: statetype;
begin

    -- state register
    process(clk, reset) begin
        if reset then state <= S0;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    -- next state logic
    nextstate <= S1 when state = S0 else
                S2 when state = S1 else
                S0;

    -- output logic
    y <= '1' when state = S0 else '0';
end;

```

This example defines a new *enumeration* data type, `statetype`, with three possibilities: S0, S1, and S2. `state` and `nextstate` are `statetype` signals. By using an enumeration instead of choosing the state encoding, VHDL frees the synthesizer to explore various state encodings to choose the best one.

In the HDL above, the output, `y`, is 1 when the state is S0. The inequality comparison uses `/=`. To produce an output of 1 when the state is anything but S0, change the comparison to `state /= S0`.

HDL Example 4.30 describes the divide-by-3 FSM from [Section 3.4.2](#). It provides an asynchronous reset to initialize the FSM. The state register uses the ordinary idiom for flip-flops. The next state and output logic blocks are combinational.

Synthesis tools produce just a block diagram and state transition diagram for state machines; they do not show the logic gates or the inputs

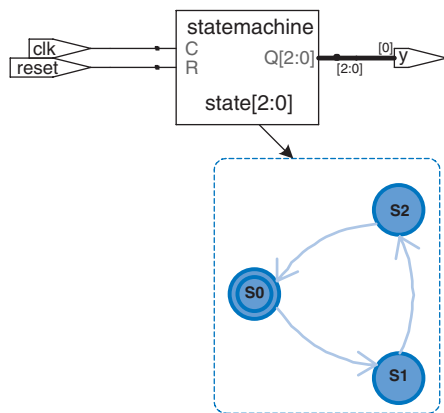


Figure 4.25 divideby3fsm
synthesized circuit

and outputs on the arcs and states. Therefore, be careful that you have specified the FSM correctly in your HDL code. The state transition diagram in Figure 4.25 for the divide-by-3 FSM is analogous to the diagram in Figure 3.28(b). The double circle indicates that S0 is the reset state. Gate-level implementations of the divide-by-3 FSM were shown in Section 3.4.2.

Notice that the states are named with an enumeration data type rather than by referring to them as binary values. This makes the code more readable and easier to change.

If, for some reason, we had wanted the output to be HIGH in states S0 and S1, the output logic would be modified as follows.

Notice that the synthesis tool uses a 3-bit encoding (Q[2:0]) instead of the 2-bit encoding suggested in the SystemVerilog code.

SystemVerilog	VHDL
<pre>// output logic assign y = (state == S0 state == S1);</pre>	<pre>-- output logic y <= '1' when (state = S0 or state = S1) else '0';</pre>

The next two examples describe the snail pattern recognizer FSM from Section 3.4.3. The code shows how to use case and if statements to handle next state and output logic that depend on the inputs as well as the current state. We show both Moore and Mealy modules. In the Moore machine (HDL Example 4.31), the output depends only on the current state, whereas in the Mealy machine (HDL Example 4.32), the output logic depends on both the current state and inputs.

HDL Example 4.31 PATTERN RECOGNIZER MOORE FSM**SystemVerilog**

```

module patternMoore(input  logic clk,
                   input  logic reset,
                   input  logic a,
                   output logic y);

    typedef enum logic [1:0] {S0, S1, S2} statetype;
    statetype state, nextstate;

    // state register
    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else      state <= nextstate;

    // next state logic
    always_comb
        case (state)
            S0: if (a) nextstate = S0;
                else nextstate = S1;
            S1: if (a) nextstate = S2;
                else nextstate = S1;
            S2: if (a) nextstate = S0;
                else nextstate = S1;
            default: nextstate = S0;
        endcase

    // output logic
    assign y = (state == S2);
endmodule

```

Note how nonblocking assignments ($\leq$) are used in the state register to describe sequential logic, whereas blocking assignments ($=$) are used in the next state logic to describe combinational logic.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity patternMoore is
    port(clk, reset: in  STD_LOGIC;
         a:      in  STD_LOGIC;
         y:      out STD_LOGIC);
end;

architecture synth of patternMoore is
    type statetype is (S0, S1, S2);
    signal state, nextstate: statetype;
begin
    -- state register
    process(clk, reset) begin
        if reset then
            state <= S0;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    -- next state logic
    process(all) begin
        case state is
            when S0 =>
                if a then nextstate <= S0;
                else      nextstate <= S1;
                end if;
            when S1 =>
                if a then nextstate <= S2;
                else      nextstate <= S1;
                end if;
            when S2 =>
                if a then nextstate <= S0;
                else      nextstate <= S1;
                end if;
            when others =>
                nextstate <= S0;
        end case;
    end process;

    --output logic
    y <= '1' when state = S2 else '0';
end;

```

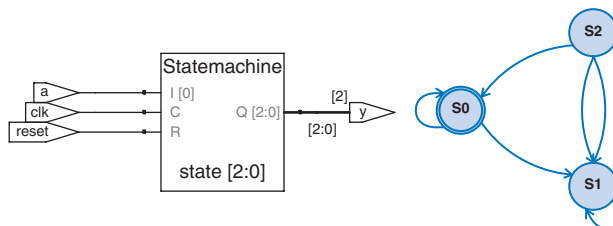


Figure 4.26 patternMoore synthesized circuit

HDL Example 4.32 PATTERN RECOGNIZER MEALY FSM**SystemVerilog**

```

module patternMealy(input logic clk,
                   input logic reset,
                   input logic a,
                   output logic y);

typedef enum logic {S0, S1} statetype;
statetype state, nextstate;

// state register
always_ff @(posedge clk, posedge reset)
    if (reset) state <= S0;
    else      state <= nextstate;

// next state logic
always_comb
    case (state)
        S0: if (a) nextstate = S0;
            else nextstate = S1;
        S1: if (a) nextstate = S0;
            else nextstate = S1;
        default: nextstate = S0;
    endcase

// output logic
assign y = (a & state == S1);
endmodule

```

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

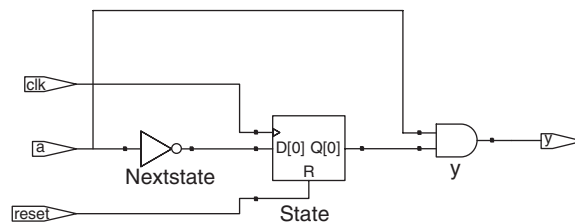
entity patternMealy is
    port (clk, reset: in  STD_LOGIC;
          a:          in  STD_LOGIC;
          y:          out STD_LOGIC);
end;

architecture synth of patternMealy is
    type statetype is (S0, S1);
    signal state, nextstate: statetype;
begin
    -- state register
    process (clk, reset) begin
        if reset then
            state <= S0;
        elsif rising_edge (clk) then
            state <= nextstate;
        end if;
    end process;

    -- next state logic
    process (all) begin
        case state is
            when S0 =>
                if a then nextstate <= S0;
                else      nextstate <= S1;
                end if;
            when S1 =>
                if a then nextstate <= S0;
                else      nextstate <= S1;
                end if;
            when others =>
                nextstate <= S0;
        end case;
    end process;

    -- output logic
    y <= '1' when (a = '1' and state = S1) else '0';
end;

```

**Figure 4.27** patternMealy synthesized circuit**4.7 DATA TYPES***

This section explains some subtleties about SystemVerilog and VHDL types in more depth.

4.7.1 SystemVerilog

Prior to SystemVerilog, Verilog primarily used two types: `reg` and `wire`. Despite its name, a `reg` signal might or might not be associated with a register. This was a great source of confusion for those learning the language. SystemVerilog introduced the `logic` type to eliminate the confusion; hence, this book emphasizes the `logic` type. This section explains the `reg` and `wire` types in more detail for those who need to read old Verilog code.

In Verilog, if a signal appears on the left-hand side of `<=` or `=` in an `always` block, it must be declared as `reg`. Otherwise, it should be declared as `wire`. Hence, a `reg` signal might be the output of a flip-flop, a latch, or combinational logic, depending on the sensitivity list and statement of an `always` block.

Input and output ports default to the `wire` type unless their type is explicitly defined as `reg`. The following example shows how a flip-flop is described in conventional Verilog. Note that `clk` and `d` default to `wire`, while `q` is explicitly defined as `reg` because it appears on the left-hand side of `<=` in the `always` block.

```
module flop(input          clk,
            input          [3:0] d,
            output reg [3:0] q);

    always @(posedge clk)
        q <= d;
endmodule
```

SystemVerilog introduces the `logic` type. `logic` is a synonym for `reg` and avoids misleading users about whether it is actually a flip-flop. Moreover, SystemVerilog relaxes the rules on `assign` statements and hierarchical port instantiations so that `logic` can be used outside `always` blocks where a `wire` traditionally would have been required. Thus, nearly all SystemVerilog signals can be `logic`. The exception is that signals with multiple drivers (e.g., a tristate bus) must be declared as a net, as described in [HDL Example 4.10](#). This rule allows SystemVerilog to generate an error message rather than an `x` value when a `logic` signal is accidentally connected to multiple drivers.

The most common type of net is called a `wire` or `tri`. These two types are synonymous, but `wire` is conventionally used when a single driver is present and `tri` is used when multiple drivers are present. Thus, `wire` is obsolete in SystemVerilog because `logic` is preferred for signals with a single driver.

When a `tri` net is driven to a single value by one or more drivers, it takes on that value. When it is undriven, it floats (`z`). When it is driven to a different value (`0`, `1`, or `x`) by multiple drivers, it is in contention (`x`).

There are other net types that resolve differently when undriven or driven by multiple sources. These other types are rarely used but may be substituted anywhere a `tri` net would normally appear (e.g., for signals with multiple drivers). Each is described in [Table 4.7](#).

Table 4.7 Net Resolution

Net Type	No Driver	Conflicting Drivers
tri	z	x
triereg	previous value	x
triand	z	0 if there are any 0
trior	z	1 if there are any 1
tri0	0	x
tri1	1	x

4.7.2 VHDL

Unlike SystemVerilog, VHDL enforces a strict data typing system that can protect the user from some errors but that is also clumsy at times.

Despite its fundamental importance, the `STD_LOGIC` type is not built into VHDL. Instead, it is part of the `IEEE.STD_LOGIC_1164` library. Thus, every file must contain the library statements shown in the previous examples.

Moreover, `IEEE.STD_LOGIC_1164` lacks basic operations such as addition, comparison, shifts, and conversion to integers for the `STD_LOGIC_VECTOR` data. These were finally added to the VHDL 2008 standard in the `IEEE.NUMERIC_STD_UNSIGNED` library.

VHDL also has a `BOOLEAN` type with two values: `true` and `false`. `BOOLEAN` values are returned by comparisons (such as the equality comparison, `s='0'`) and are used in conditional statements, such as `when` and `if`. Despite the temptation to believe a `BOOLEAN true` value should be equivalent to a `STD_LOGIC '1'` and `BOOLEAN false` should mean `STD_LOGIC '0'`, these types were not interchangeable prior to VHDL 2008. For example, in old VHDL code, one must write

```
y <= d1 when (s='1') else d0;
```

while in VHDL 2008, the `when` statement automatically converts `s` from `STD_LOGIC` to `BOOLEAN` so one can simply write

```
y <= d1 when s else d0;
```

Even in VHDL 2008, it is still necessary to write

```
q <= '1' when (state=S2) else '0';
```

instead of

```
q <= (state=S2);
```

because (state=S2) returns a BOOLEAN result, which cannot be directly assigned to the STD_LOGIC signal y.

Although we do not declare any signals to be BOOLEAN, they are automatically implied by comparisons and used by conditional statements. Similarly, VHDL has an INTEGER type that represents both positive and negative integers. Signals of type INTEGER span at least the values $-(2^{31} - 1)$ to $2^{31} - 1$. Integer values are used as indices of busses. For example, in the statement

```
y <= a(3) and a(2) and a(1) and a(0);
```

0, 1, 2, and 3 are integers serving as an index to choose bits of the a signal. We cannot directly index a bus with a STD_LOGIC or STD_LOGIC_VECTOR signal. Instead, we must convert the signal to an INTEGER. This is demonstrated in the example below for an 8:1 multiplexer that selects one bit from a vector using a 3-bit index. The TO_INTEGER function is defined in the IEEE.NUMERIC_STD_UNSIGNED library and performs the conversion from STD_LOGIC_VECTOR to INTEGER for positive (unsigned) values.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;
entity mux8 is
  port(d: in  STD_LOGIC_VECTOR(7 downto 0);
        s: in  STD_LOGIC_VECTOR(2 downto 0);
        y: out STD_LOGIC);
end;
architecture synth of mux8 is
begin
  y <= d(TO_INTEGER(s));
end;
```

VHDL is also strict about out ports being exclusively for output. For example, the following code for two- and three-input AND gates is illegal VHDL because v is an output and is also used to compute w.

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
entity and23 is
  port(a, b, c: in STD_LOGIC;
        v, w: out  STD_LOGIC);
end;
architecture synth of and23 is
begin
  v <= a and b;
  w <= v and c;
end;
```

VHDL defines a special port type, buffer, to solve this problem. A signal connected to a buffer port behaves as an output but may also be used within the module. The corrected entity definition follows. Verilog

and SystemVerilog do not have this limitation and do not require buffer ports. VHDL 2008 eliminates this restriction by allowing out ports to be readable.

```
entity and23 is
  port(a, b, c: in STD_LOGIC;
        v: buffer    STD_LOGIC;
        w: out      STD_LOGIC);
end;
```

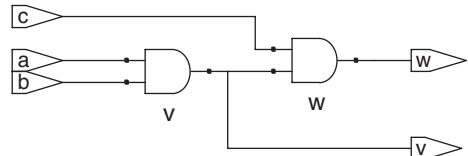


Figure 4.28 and23 synthesized circuit

Most operations such as addition, subtraction, and Boolean logic are identical whether a number is signed or unsigned. However, magnitude comparison, multiplication, and arithmetic right shifts are performed differently for signed two's complement numbers than for unsigned binary numbers. These operations will be examined in Chapter 5. [HDL Example 4.33](#) describes how to indicate that a signal represents a signed number.

HDL Example 4.33 (a) UNSIGNED MULTIPLIER (b) SIGNED MULTIPLIER

SystemVerilog

```
// 4.33(a): unsigned multiplier
module multiplier(input  logic [3:0] a, b,
                  output logic [7:0] y);
  assign y = a * b;
endmodule

// 4.33(b): signed multiplier
module multiplier(input  logic signed [3:0] a, b,
                  output logic signed [7:0] y);
  assign y = a * b;
endmodule
```

In SystemVerilog, signals are considered unsigned by default. Adding the signed modifier (e.g., `logic signed [3:0] a`) causes the signal `a` to be treated as a signed—that is, two's complement—number.

VHDL

```
-- 4.33(a): unsigned multiplier
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity multiplier is
  port(a, b: in  STD_LOGIC_VECTOR(3 downto 0);
        y:   out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture synth of multiplier is
begin
  y <= a * b;
end;
```

VHDL uses the `NUMERIC_STD_UNSIGNED` library to perform arithmetic and comparison operations on `STD_LOGIC_VECTORS`. The vectors are treated as unsigned.

```
use IEEE.NUMERIC_STD_UNSIGNED.all;
```

VHDL also defines `UNSIGNED` and `SIGNED` data types in the `IEEE.NUMERIC_STD` library, but these involve type conversions beyond the scope of this chapter.

4.8 PARAMETERIZED MODULES*

So far, all of our modules have had fixed-width inputs and outputs. For example, we had to define separate modules for 4- and 8-bit-wide 2:1 multiplexers. HDLs permit variable bit widths using parameterized modules.

[HDL Example 4.34](#) declares a parameterized 2:1 multiplexer with a default width of 8, then uses it to create 8- and 12-bit 4:1 multiplexers.

HDL Example 4.34 PARAMETERIZED N-BIT 2:1 MULTIPLEXERS**SystemVerilog**

```

module mux2
  #(parameter width = 8)
  (input  logic [width-1:0] d0, d1,
   input  logic      s,
   output logic [width-1:0] y);

  assign y = s ? d1 : d0;
endmodule

```

SystemVerilog allows a `#(parameter...)` statement before the inputs and outputs to define parameters. The parameter statement includes a default value (8) of the parameter, in this case called `width`. The number of bits in the inputs and outputs can depend on this parameter.

```

module mux4_8(input  logic [7:0] d0, d1, d2, d3,
              input  logic [1:0] s,
              output logic [7:0] y);

  logic [7:0] low, hi;

  mux2 lowmux(d0, d1, s[0], low);
  mux2 himux(d2, d3, s[0], hi);
  mux2 outmux(low, hi, s[1], y);
endmodule

```

The 8-bit 4:1 multiplexer instantiates three 2:1 multiplexers using their default widths.

In contrast, a 12-bit 4:1 multiplexer, `mux4_12`, would need to override the default width using `#( )` before the instance name, as shown below.

```

module mux4_12(input  logic [11:0] d0, d1, d2, d3,
               input  logic [1:0] s,
               output logic [11:0] y);

  logic [11:0] low, hi;

  mux2 #(12) lowmux(d0, d1, s[0], low);
  mux2 #(12) himux(d2, d3, s[0], hi);
  mux2 #(12) outmux(low, hi, s[1], y);
endmodule

```

Do not confuse the use of the `#` sign indicating delays with the use of `#(...)` in defining and overriding parameters.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux2 is
  generic(width: integer := 8);
  port(d0,
       d1: in  STD_LOGIC_VECTOR(width-1 downto 0);
       s: in  STD_LOGIC;
       y: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture synth of mux2 is
begin
  y <= d1 when s else d0;
end;

```

The generic statement includes a default value (8) of `width`. The value is an integer.

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity mux4_8 is
  port(d0, d1, d2,
       d3: in  STD_LOGIC_VECTOR(7 downto 0);
       s: in  STD_LOGIC_VECTOR(1 downto 0);
       y: out STD_LOGIC_VECTOR(7 downto 0));
end;

architecture struct of mux4_8 is
  component mux2
    generic(width: integer := 8);
    port(d0,
         d1: in  STD_LOGIC_VECTOR(width-1 downto 0);
         s: in  STD_LOGIC;
         y: out STD_LOGIC_VECTOR(width-1 downto 0));
  end component;
  signal low, hi: STD_LOGIC_VECTOR(7 downto 0);
begin
  lowmux: mux2 port map(d0, d1, s(0), low);
  himux:  mux2 port map(d2, d3, s(0), hi);
  outmux: mux2 port map(low, hi, s(1), y);
end;

```

The 8-bit 4:1 multiplexer, `mux4_8`, instantiates three 2:1 multiplexers, using their default widths.

In contrast, a 12-bit 4:1 multiplexer, `mux4_12`, would need to override the default width using generic `map`, as shown below.

```

lowmux: mux2 generic map(12)
  port map(d0, d1, s(0), low);
himux:  mux2 generic map(12)
  port map(d2, d3, s(0), hi);
outmux: mux2 generic map(12)
  port map(low, hi, s(1), y);

```

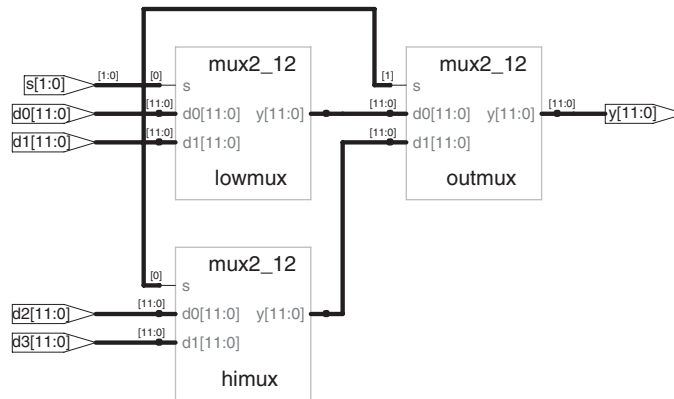


Figure 4.29 mux4_12 synthesized circuit

HDL Example 4.35 PARAMETERIZED $N:2^N$ DECODER**SystemVerilog**

```

module decoder
#(parameter N = 3)
  (input logic [N-1:0] a,
   output logic [2**N-1:0] y);

  always_comb
  begin
    y = 0;
    y[a] = 1;
  end
endmodule

```

2^{**N} indicates 2^N .

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity decoder is
  generic(N: integer := 3);
  port(a: in  STD_LOGIC_VECTOR(N-1 downto 0);
       y: out STD_LOGIC_VECTOR(2**N-1 downto 0));
end;

```

```

architecture synth of decoder is
begin
  process(all)
  begin
    y <= (OTHERS => '0');
    y(TO_INTEGER(a)) <= '1';
  end process;
end;

```

2^{**N} indicates 2^N .

HDL Example 4.35 shows a decoder, which is an even better application of parameterized modules. A large $N:2^N$ decoder is cumbersome to specify with case statements, but easy using parameterized code that simply sets the appropriate output bit to 1. Specifically, the decoder uses blocking assignments to set all the bits to 0, then changes the appropriate bit to 1.

HDLs also provide generate statements to produce a variable amount of hardware, depending on the value of a parameter. generate supports for loops and if statements to determine how many of what types of hardware to produce. HDL Example 4.36 demonstrates how to use generate statements to produce an N -input AND function from a

cascade of two-input AND gates. Of course, a reduction operator would be cleaner and simpler for this application, but the example illustrates the general principle of hardware generators.

Use generate statements with caution; it is easy to produce a large amount of hardware unintentionally!

HDL Example 4.36 **PARAMETERIZED N-INPUT AND GATE**

SystemVerilog

```
module andN
#(parameter width = 8)
(input logic [width-1:0] a,
 output logic y);

genvar i;
logic [width-1:0] x;

generate
  assign x[0] = a[0];
  for(i=1; i<width; i=i+1) begin: forloop
    assign x[i] = a[i] & x[i-1];
  end
endgenerate

assign y = x[width-1];
endmodule
```

The for statement loops through $i = 1, 2, \dots, \text{width}-1$ to produce many consecutive AND gates. The begin in a generate for loop must be followed by a : and an arbitrary label (forloop, in this case).

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity andN is
  generic(width: integer := 8);
  port(a: in STD_LOGIC_VECTOR(width-1 downto 0);
        y: out STD_LOGIC);
end;

architecture synth of andN is
  signal x: STD_LOGIC_VECTOR(width-1 downto 0);
begin
  x(0) <= a(0);
  gen: for i in 1 to width-1 generate
    x(i) <= a(i) and x(i-1);
  end generate;
  y <= x(width-1);
end;
```

The generate loop variable i does not need to be declared.

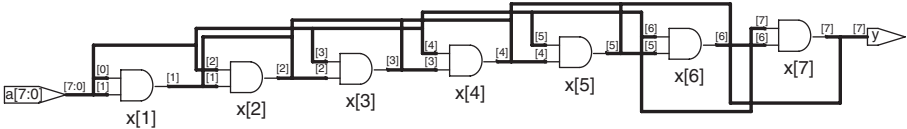


Figure 4.30 **andN synthesized circuit**

Some tools also call the module to be tested the *unit under test* (UUT).

4.9 TESTBENCHES

A *testbench* is an HDL module that is used to test another module, called the *device under test* (DUT). The testbench contains statements to apply inputs to the DUT and, ideally, to check that the correct outputs are produced. The input and desired output patterns are called *test vectors*.

Consider testing the *sillyfunction* module from [Section 4.1.1](#) that computes $y = \overline{a}\overline{b}\overline{c} + a\overline{b}\overline{c} + a\overline{b}c$. This is a simple module, so we can perform exhaustive testing by applying all eight possible test vectors.

HDL Example 4.37 TESTBENCH**SystemVerilog**

```

module testbench1();
    logic a, b, c, y;

    // instantiate device under test
    sillyfunction dut(a, b, c, y);

    // apply inputs one at a time
    initial begin
        a = 0; b = 0; c = 0; #10;
        c = 1;          #10;
        b = 1; c = 0;    #10;
        c = 1;          #10;
        a = 1; b = 0; c = 0; #10;
        c = 1;          #10;
        b = 1; c = 0;    #10;
        c = 1;          #10;
    end
endmodule

```

The `initial` statement executes the statements in its body at the start of simulation. In this case, it first applies the input pattern 000 and waits for 10 time units. It then applies 001 and waits 10 more units, and so forth until all eight possible inputs have been applied. `initial` statements should be used only in testbenches for simulation, not in modules intended to be synthesized into actual hardware. Hardware has no way of magically executing a sequence of special steps when it is first turned on.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity testbench1 is -- no inputs or outputs
end;

architecture sim of testbench1 is
    component sillyfunction
        port(a, b, c: in  STD_LOGIC;
             y:      out STD_LOGIC);
    end component;
    signal a, b, c, y: STD_LOGIC;
begin
    -- instantiate device under test
    dut: sillyfunction port map(a, b, c, y);

    -- apply inputs one at a time
    process begin
        a <= '0'; b <= '0'; c <= '0'; wait for 10 ns;
        c <= '1';          wait for 10 ns;
        b <= '1'; c <= '0';    wait for 10 ns;
        c <= '1';          wait for 10 ns;
        a <= '1'; b <= '0'; c <= '0'; wait for 10 ns;
        c <= '1';          wait for 10 ns;
        b <= '1'; c <= '0';    wait for 10 ns;
        c <= '1';          wait for 10 ns;
        wait; --wait forever
    end process;
end;

```

The `process` statement first applies the input pattern 000 and waits for 10 ns. It then applies 001 and waits 10 more ns, and so forth until all eight possible inputs have been applied.

At the end, the process waits indefinitely. Otherwise, the process would begin again, repeatedly applying the pattern of test vectors.

HDL Example 4.37 demonstrates a simple testbench. It instantiates the DUT, then applies the inputs. Blocking assignments and delays are used to apply the inputs in the appropriate order. The user must view the results of the simulation and verify by inspection that the correct outputs are produced. Testbenches are simulated the same as other HDL modules. However, they are not synthesizable.

Checking for correct outputs is tedious and error-prone. Moreover, determining the correct outputs is much easier when the design is fresh in your mind. If you make minor changes and need to retest weeks later, determining the correct outputs becomes a hassle. A much better approach is to write a self-checking testbench, shown in **HDL Example 4.38**.

Writing code for each test vector also becomes tedious, especially for modules that require a large number of vectors. An even better

HDL Example 4.38 SELF-CHECKING TESTBENCH**SystemVerilog**

```

module testbench2();
  logic a, b, c, y;

  // instantiate device under test
  sillyfunction dut(a, b, c, y);

  // apply inputs one at a time
  // checking results
  initial begin
    a = 0; b = 0; c = 0; #10;
    assert (y === 1) else $error("000 failed.");
    c = 1; #10;
    assert (y === 0) else $error("001 failed.");
    b = 1; c = 0; #10;
    assert (y === 0) else $error("010 failed.");
    c = 1; #10;
    assert (y === 0) else $error("011 failed.");
    a = 1; b = 0; c = 0; #10;
    assert (y === 1) else $error("100 failed.");
    c = 1; #10;
    assert (y === 1) else $error("101 failed.");
    b = 1; c = 0; #10;
    assert (y === 0) else $error("110 failed.");
    c = 1; #10;
    assert (y === 0) else $error("111 failed.");
  end
endmodule

```

The SystemVerilog `assert` statement checks whether a specified condition is true. If not, it executes the `else` statement. The `$error` system task in the `else` statement prints an error message describing the assertion failure. `assert` is ignored during synthesis.

In SystemVerilog, comparison using `==` or `!=` is effective between signals that do not take on the values of `x` and `z`. Testbenches use the `===` and `!==` operators for comparisons of equality and inequality, respectively, because these operators work correctly with operands that could be `x` or `z`.

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity testbench2 is -- no inputs or outputs
end;

architecture sim of testbench2 is
  component sillyfunction
    port(a, b, c: in STD_LOGIC;
         y: out STD_LOGIC);
  end component;
  signal a, b, c, y: STD_LOGIC;
begin
  -- instantiate device under test
  dut: sillyfunction port map(a, b, c, y);

  -- apply inputs one at a time
  -- checking results
  process begin
    a <= '0'; b <= '0'; c <= '0'; wait for 10 ns;
    assert y = '1' report "000 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "001 failed.";
    b <= '1'; c <= '0'; wait for 10 ns;
    assert y = '0' report "010 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "011 failed.";
    a <= '1'; b <= '0'; c <= '0'; wait for 10 ns;
    assert y = '1' report "100 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '1' report "101 failed.";
    b <= '1'; c <= '0'; wait for 10 ns;
    assert y = '0' report "110 failed.";
    c <= '1'; wait for 10 ns;
    assert y = '0' report "111 failed.";
    wait; -- wait forever
  end process;
end;

```

The `assert` statement checks a condition and prints the message given in the `report` clause if the condition is not satisfied. `assert` is meaningful only in simulation, not in synthesis.

approach is to place the test vectors in a separate file. The testbench simply reads the test vectors from the file, applies the input test vector to the DUT, waits, checks that the output values from the DUT match the output vector, and repeats until reaching the end of the test vectors file.

HDL Example 4.39 demonstrates such a testbench. The testbench generates a clock using an `always/process` statement with no sensitivity list, so that it is continuously reevaluated. At the beginning of the simulation, it reads the test vectors from a text file and pulses `reset` for two cycles. Although the clock and `reset` aren't necessary to test combinational logic, they are included because they would be important when

testing a sequential DUT. `example.txt` is a text file containing the *test vectors*, the inputs and expected output written in binary:

```
000_1
001_0
010_0
011_0
100_1
101_1
110_0
111_0
```

HDL Example 4.39 TESTBENCH WITH TEST VECTOR FILE

SystemVerilog

```
module testbench3();
    logic      clk, reset;
    logic      a, b, c, y, yexpected;
    logic [31:0] vectornum, errors;
    logic [3:0] testvectors[10000:0];

    // instantiate device under test
    sillyfunction dut(a, b, c, y);

    // generate clock
    always
    begin
        clk = 1; #5; clk = 0; #5;
    end

    // at start of test, load vectors
    // and pulse reset
    initial
    begin
        $readmemb("example.txt", testvectors);
        vectornum = 0; errors = 0;
        reset = 1; #22; reset = 0;
    end

    // apply test vectors on rising edge of clk
    always @(posedge clk)
    begin
        #1; (a, b, c, yexpected) = testvectors[vectornum];
    end

    // check results on falling edge of clk
    always @(negedge clk)
    if (~reset) begin // skip during reset
        if (y !== yexpected) begin // check result
            $display("Error: inputs = %b", {a, b, c});
            $display(" outputs = %b (%b expected)", y, yexpected);
            errors = errors + 1;
        end
        vectornum = vectornum + 1;
        if (testvectors[vectornum] === 4'bx) begin
            $display("%d tests completed with %d errors",
                vectornum, errors);
        end
        $stop;
    end
end
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_TEXTIO.ALL; use STD.TEXTIO.all;

entity testbench3 is -- no inputs or outputs
end;

architecture sim of testbench3 is
    component sillyfunction
        port(a, b, c: in  STD_LOGIC;
             y:      out STD_LOGIC);
    end component;
    signal a, b, c, y:  STD_LOGIC;
    signal y_expected: STD_LOGIC;
    signal clk, reset: STD_LOGIC;
begin
    -- instantiate device under test
    dut: sillyfunction port map(a, b, c, y);

    -- generate clock
    process begin
        clk <= '1'; wait for 5 ns;
        clk <= '0'; wait for 5 ns;
    end process;

    -- at start of test, pulse reset
    process begin
        reset <= '1'; wait for 27 ns; reset <= '0';
        wait;
    end process;

    -- run tests
    process is
        file tv: text;
        variable L: line;
        variable vector_in: std_logic_vector(2 downto 0);
        variable dummy: character;
        variable vector_out: std_logic;
        variable vectornum: integer := 0;
        variable errors: integer := 0;
    begin
        FILE_OPEN(tv, "example.txt", READ_MODE);
        while not endfile(tv) loop

            -- change vectors on rising edge
            wait until rising_edge(clk);

            -- read the next line of testvectors and split into pieces
            readline(tv, L);
            read(L, vector_in);
            read(L, dummy); -- skip over underscore
```

`$readmemb` reads a file of binary numbers into the `testvectors` array. `$readmemh` is similar but reads a file of hexadecimal numbers.

The next block of code waits one time unit after the rising edge of the clock (to avoid any confusion if clock and data change simultaneously), then sets the three inputs and the expected output based on the four bits in the current test vector.

The testbench compares the generated output, `y`, with the expected output, `yexpected`, and prints an error if they don't match. `%b` and `%d` indicate to print the values in binary and decimal, respectively. `$display` is a system task to print in the simulator window. For example, `$display ("%b %b", y, yexpected);` prints the two values, `y` and `yexpected`, in binary. `%h` prints a value in hexadecimal.

This process repeats until there are no more valid test vectors in the `testvectors` array. `$stop` stops the simulation.

Note that even though the SystemVerilog module supports up to 10,001 test vectors, it will terminate the simulation after executing the eight vectors in the file.

```
read(L, vector_out);
(a, b, c) <= vector_in(2 downto 0) after 1 ns;
y_expected <= vector_out after 1 ns;
```

```
-- check results on falling edge
wait until falling_edge(clk);
```

```
if y /= y_expected then
    report "Error: y = " & std_logic'image(y);
    errors := errors + 1;
end if;
```

```
vectornum := vectornum + 1;
end loop;
```

```
-- summarize results at end of simulation
if (errors = 0) then
    report "NO ERRORS -- " &
        integer'image(vectornum) &
        " tests completed successfully."
        severity failure;
```

```
else
    report integer'image(vectornum) &
        " tests completed, errors = " &
        integer'image(errors)
        severity failure;
```

```
end if;
end process;
end;
```

The VHDL code uses file reading commands beyond the scope of this chapter, but it gives the sense of what a self-checking testbench looks like.

New inputs are applied on the rising edge of the clock, and the output is checked on the falling edge of the clock. Errors are reported as they occur. At the end of the simulation, the testbench prints the total number of test vectors applied and the number of errors detected.

The testbench in [HDL Example 4.39](#) is overkill for such a simple circuit. However, it can easily be modified to test more complex circuits by changing the `example.txt` file, instantiating the new DUT, and changing a few lines of code to set the inputs and check the outputs.

4.10 SUMMARY

Hardware description languages (HDLs) are extremely important tools for modern digital designers. Once you have learned SystemVerilog or VHDL, you will be able to specify digital systems much faster than if you had to draw the complete schematics. The debug cycle is also often much faster because modifications require code changes instead of tedious schematic rewiring. However, the debug cycle can be much *longer* using HDLs if you don't have a good idea of the hardware your code implies.

HDLs are used for both simulation and synthesis. Logic simulation is a powerful way to test a system on a computer before it is turned into hardware. Simulators let you check the values of signals inside your system that might be impossible to measure on a physical piece of hardware. Logic synthesis converts the HDL code into digital logic circuits.

The most important thing to remember when you are writing HDL code is that you are describing real hardware, not writing a computer program. The most common beginner's mistake is to write HDL code without thinking about the hardware you intend to produce. If you don't know what hardware you are implying, you are almost certain not going to get what you want. Instead, begin by sketching a block diagram of your system, identifying which portions are combinational logic, which portions are sequential circuits or finite state machines, and so forth. Then, write HDL code for each portion, using the correct idioms to imply the kind of hardware you need.

Exercises

The following exercises may be done using your favorite HDL. If you have a simulator available, test your design. Print the waveforms and explain how they prove that it works. If you have a synthesizer available, synthesize your code. Print the generated circuit diagram and explain why it matches your expectations.

Exercise 4.1 Sketch a schematic of the circuit described by the following HDL code. Simplify the schematic so that it shows a minimum number of gates.

SystemVerilog

```
module exercise1(input  logic a, b, c,
                 output logic y, z);

    assign y = a & b & c | a & b & ~c | a & ~b & c;
    assign z = a & b | ~a & ~b;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity exercise1 is
    port(a, b, c: in  STD_LOGIC;
         y, z:  out STD_LOGIC);
end;

architecture synth of exercise1 is
begin
    y <= (a and b and c) or (a and b and not c) or
        (a and not b and c);
    z <= (a and b) or (not a and not b);
end;
```

Exercise 4.2 Sketch a schematic of the circuit described by the following HDL code. Simplify the schematic so that it shows a minimum number of gates.

SystemVerilog

```
module exercise2(input  logic [3:0] a,
                 output logic [1:0] y);

    always_comb
        if      (a[0]) y = 2'b11;
        else if (a[1]) y = 2'b10;
        else if (a[2]) y = 2'b01;
        else if (a[3]) y = 2'b00;
        else          y = a[1:0];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity exercise2 is
    port(a: in  STD_LOGIC_VECTOR(3 downto 0);
         y: out STD_LOGIC_VECTOR(1 downto 0));
end;

architecture synth of exercise2 is
begin
    process(all) begin
        if    a(0) then y <= "11";
        elsif a(1) then y <= "10";
        elsif a(2) then y <= "01";
        elsif a(3) then y <= "00";
        else          y <= a(1 downto 0);
        end if;
    end process;
end;
```

Exercise 4.3 Write an HDL module that computes a four-input XOR function. The input is $a_{3:0}$ and the output is y .

Exercise 4.4 Write a self-checking testbench for Exercise 4.3. Create a test vector file containing all 16 test cases. Simulate the circuit and show that it works.

Introduce an error in the test vector file and show that the testbench reports a mismatch.

Exercise 4.5 Write an HDL module called `minority`. It receives three inputs, `a`, `b`, and `c`. It produces one output, `y`, that is TRUE if at least two of the inputs are FALSE.

Exercise 4.6 Write an HDL module for a hexadecimal seven-segment display decoder. The decoder should handle the digits A, B, C, D, E, and F, as well as 0–9.

Exercise 4.7 Write a self-checking testbench for Exercise 4.6. Create a test vector file containing all 16 test cases. Simulate the circuit and show that it works. Introduce an error in the test vector file and show that the testbench reports a mismatch.

Exercise 4.8 Write an 8:1 multiplexer module called `mux8` with inputs `s2:0`, `d0`, `d1`, `d2`, `d3`, `d4`, `d5`, `d6`, `d7`, and output `y`.

Exercise 4.9 Write a structural module to compute the logic function $y = a\bar{b} + \bar{b}\bar{c} + \bar{a}bc$ using multiplexer logic. Use the 8:1 multiplexer from Exercise 4.8.

Exercise 4.10 Repeat Exercise 4.9 using a 4:1 multiplexer and as many NOT gates as you need.

Exercise 4.11 Section 4.5.4 pointed out that a synchronizer could be correctly described with blocking assignments if the assignments were given in the proper order. Think of a simple sequential circuit that cannot be correctly described with blocking assignments, regardless of order.

Exercise 4.12 Write an HDL module for an eight-input priority circuit.

Exercise 4.13 Write an HDL module for a 2:4 decoder.

Exercise 4.14 Write an HDL module for a 6:64 decoder using three instances of the 2:4 decoders from Exercise 4.13 and a bunch of three-input AND gates.

Exercise 4.15 Write HDL modules that implement the Boolean equations from Exercise 2.13.

Exercise 4.16 Write an HDL module that implements the circuit from Exercise 2.26.

Exercise 4.17 Write an HDL module that implements the circuit from Exercise 2.27.

Exercise 4.18 Write an HDL module that implements the logic function from Exercise 2.28. Pay careful attention to how you handle don't cares.

Exercise 4.19 Write an HDL module that implements the functions from Exercise 2.35.

Exercise 4.20 Write an HDL module that implements the priority encoder from Exercise 2.36.

Exercise 4.21 Write an HDL module that implements the modified priority encoder from Exercise 2.37.

Exercise 4.22 Write an HDL module that implements the binary-to-thermometer code converter from Exercise 2.38.

Exercise 4.23 Write an HDL module implementing the days-in-month function from Question 2.2.

Exercise 4.24 Sketch the state transition diagram for the FSM described by the following HDL code.

SystemVerilog

```
module fsm2(input  logic clk, reset,
            input  logic a, b,
            output logic y);

    logic [1:0] state, nextstate;

    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;
    parameter S3 = 2'b11;

    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S0;
        else      state <= nextstate;

    always_comb
        case (state)
            S0: if (a ^ b) nextstate = S1;
                else      nextstate = S0;
            S1: if (a & b) nextstate = S2;
                else      nextstate = S0;
            S2: if (a | b) nextstate = S3;
                else      nextstate = S0;
            S3: if (a | b) nextstate = S3;
                else      nextstate = S0;
        endcase

    assign y = (state == S1) | (state == S2);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.all;

entity fsm2 is
    port(clk, reset: in  STD_LOGIC;
          a, b:      in  STD_LOGIC;
          y:         out STD_LOGIC);
end;

architecture synth of fsm2 is
    type statetype is (S0, S1, S2, S3);
    signal state, nextstate: statetype;
begin
    process(clk, reset) begin
        if reset then state <= S0;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    process(all) begin
        case state is
            when S0 => if (a xor b) then
                            nextstate <= S1;
                        else nextstate <= S0;
                        end if;
            when S1 => if (a and b) then
                            nextstate <= S2;
                        else nextstate <= S0;
                        end if;
            when S2 => if (a or b) then
                            nextstate <= S3;
                        else nextstate <= S0;
                        end if;
            when S3 => if (a or b) then
                            nextstate <= S3;
                        else nextstate <= S0;
                        end if;
        end case;
    end process;

    y <= '1' when ((state = S1) or (state = S2))
        else '0';
end;
```


Exercise 4.25 Sketch the state transition diagram for the FSM described by the following HDL code. An FSM of this nature is used in a branch predictor on some microprocessors.

SystemVerilog

```
module fsm1(input  logic clk, reset,
            input  logic taken, back,
            output logic predicttaken);

    logic [4:0] state, nextstate;

    parameter S0 = 5'b00001;
    parameter S1 = 5'b00010;
    parameter S2 = 5'b00100;
    parameter S3 = 5'b01000;
    parameter S4 = 5'b10000;

    always_ff @(posedge clk, posedge reset)
        if (reset) state <= S2;
        else      state <= nextstate;

    always_comb
        case (state)
            S0: if (taken) nextstate = S1;
                else      nextstate = S0;
            S1: if (taken) nextstate = S2;
                else      nextstate = S0;
            S2: if (taken) nextstate = S3;
                else      nextstate = S1;
            S3: if (taken) nextstate = S4;
                else      nextstate = S2;
            S4: if (taken) nextstate = S4;
                else      nextstate = S3;
            default:      nextstate = S2;
        endcase

    assign predicttaken = (state == S4) |
                          (state == S3) |
                          (state == S2 & back);

endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164. all;

entity fsm1 is
    port(clk, reset:  in  STD_LOGIC;
         taken, back: in  STD_LOGIC;
         predicttaken: out STD_LOGIC);
end;

architecture synth of fsm1 is
    type statetype is (S0, S1, S2, S3, S4);
    signal state, nextstate: statetype;
begin
    process(clk, reset) begin
        if reset then state <= S2;
        elsif rising_edge(clk) then
            state <= nextstate;
        end if;
    end process;

    process(all) begin
        case state is
            when S0 => if taken then
                            nextstate <= S1;
                        else nextstate <= S0;
                        end if;
            when S1 => if taken then
                            nextstate <= S2;
                        else nextstate <= S0;
                        end if;
            when S2 => if taken then
                            nextstate <= S3;
                        else nextstate <= S1;
                        end if;
            when S3 => if taken then
                            nextstate <= S4;
                        else nextstate <= S2;
                        end if;
            when S4 => if taken then
                            nextstate <= S4;
                        else nextstate <= S3;
                        end if;
            when others => nextstate <= S2;
        end case;
    end process;

    -- output logic
    predicttaken <= '1' when
        ((state = S4) or (state = S3) or
         (state = S2 and back = '1'))
    else '0';
end;
```

Exercise 4.26 Write an HDL module for an SR latch.

Exercise 4.27 Write an HDL module for a *JK flip-flop*. The flip-flop has inputs, *clk*, *J*, and *K*, and output *Q*. On the rising edge of the clock, *Q* keeps its old value if $J = K = 0$. It sets *Q* to 1 if $J = 1$, resets *Q* to 0 if $K = 1$, and inverts *Q* if $J = K = 1$.

Exercise 4.28 Write an HDL module for the latch from Figure 3.18. Use one assignment statement for each gate. Specify delays of 1 unit or 1 ns to each gate. Simulate the latch and show that it operates correctly. Then, increase the inverter delay. How long does the delay have to be before a race condition causes the latch to malfunction?

Exercise 4.29 Write an HDL module for the traffic light controller from Section 3.4.1.

Exercise 4.30 Write three HDL modules for the factored parade mode traffic light controller from Example 3.8. The modules should be called `controller`, `mode`, and `lights`, and they should have the inputs and outputs shown in Figure 3.33(b).

Exercise 4.31 Write an HDL module describing the circuit in Figure 3.42.

Exercise 4.32 Write an HDL module for the FSM with the state transition diagram given in Figure 3.69 from Exercise 3.22.

Exercise 4.33 Write an HDL module for the FSM with the state transition diagram given in Figure 3.70 from Exercise 3.23.

Exercise 4.34 Write an HDL module for the improved traffic light controller from Exercise 3.24.

Exercise 4.35 Write an HDL module for the daughter snail from Exercise 3.25.

Exercise 4.36 Write an HDL module for the soda machine dispenser from Exercise 3.26.

Exercise 4.37 Write an HDL module for the Gray code counter from Exercise 3.27.

Exercise 4.38 Write an HDL module for the UP/DOWN Gray code counter from Exercise 3.28.

Exercise 4.39 Write an HDL module for the FSM from Exercise 3.29.

Exercise 4.40 Write an HDL module for the FSM from Exercise 3.30.

Exercise 4.41 Write an HDL module for the serial two's complementer from Question 3.2.

Exercise 4.42 Write an HDL module for the circuit in Exercise 3.31.

Exercise 4.43 Write an HDL module for the circuit in Exercise 3.32.

Exercise 4.44 Write an HDL module for the circuit in Exercise 3.33.

Exercise 4.45 Write an HDL module for the circuit in Exercise 3.34. You may use the full adder from [Section 4.2.5](#).

SystemVerilog Exercises

The following exercises are specific to SystemVerilog.

Exercise 4.46 What does it mean for a signal to be declared `tri` in SystemVerilog?

Exercise 4.47 Rewrite the `syncbad` module from [HDL Example 4.29](#). Use nonblocking assignments, but change the code to produce a correct synchronizer with two flip-flops.

Exercise 4.48 Consider the following two SystemVerilog modules. Do they have the same function? Sketch the hardware each one implies.

```
module code1(input  logic clk, a, b, c,
             output logic y);
    logic x;
    always_ff @(posedge clk) begin
        x <= a & b;
        y <= x | c;
    end
endmodule

module code2(input  logic a, b, c, clk,
             output logic y);
    logic x;
    always_ff @(posedge clk) begin
        y <= x | c;
        x <= a & b;
    end
endmodule
```

Exercise 4.49 Repeat Exercise 4.48 if the `<=` is replaced by `=` in every assignment.

Exercise 4.50 The following SystemVerilog modules show errors that the authors have seen students make in the laboratory. Explain the error in each module and show how to fix it.

- (a)

```
module latch(input logic      clk,
             input logic [3:0] d,
             output reg  [3:0] q);
    always @(clk)
        if (clk) q <= d;
endmodule
```
- (b)

```
module gates(input  logic [3:0] a, b,
             output logic [3:0] y1, y2, y3, y4, y5);
    always @(a)
    begin
        y1 = a & b;
        y2 = a | b;
        y3 = a ^ b;
        y4 = ~(a & b);
        y5 = ~(a | b);
    end
endmodule
```
- (c)

```
module mux2(input  logic [3:0] d0, d1,
            input  logic      s,
            output logic [3:0] y);
    always @(posedge s)
        if(s) y <= d1;
        else y <= d0;
endmodule
```
- (d)

```
module twoflops(input  logic clk,
                input  logic d0, d1,
                output logic q0, q1);
    always @(posedge clk)
        q1=d1;
        q0=d0;
endmodule
```
- (e)

```
module FSM(input  logic clk,
           input  logic a,
           output logic out1, out2);

    logic state;

    // next state logic and register (sequential)
    always_ff @(posedge clk)
        if (state==0) begin
            if (a) state <= 1;
        end else begin
            if (~a) state <= 0;
        end
end
```

```

        always_comb // output logic (combinational)
        if (state == 0) out1 = 1;
        else
            out2 = 1;
    endmodule

```

(f) module priority(input logic [3:0] a,
output logic [3:0] y);

```

    always_comb
    if      (a[3]) y = 4'b1000;
    else if (a[2]) y = 4'b0100;
    else if (a[1]) y = 4'b0010;
    else if (a[0]) y = 4'b0001;
endmodule

```

(g) module divideby3FSM(input logic clk,
input logic reset,
output logic out);

```

    logic [1:0] state, nextstate;

    parameter S0 = 2'b00;
    parameter S1 = 2'b01;
    parameter S2 = 2'b10;

    // state register
    always_ff @(posedge clk, posedge reset)
    if (reset) state <= S0;
    else      state <= nextstate;

    // next state logic
    always @(state)
    case (state)
        S0: nextstate = S1;
        S1: nextstate = S2;
        S2: nextstate = S0;
    endcase

    // output logic
    assign out = (state == S2);
endmodule

```

(h) module mux2tri(input logic [3:0] d0, d1,
input logic s,
output tri [3:0] y);
tristate t0(d0, s, y);
tristate t1(d1, s, y);
endmodule

(i) module floprsen(input logic clk,
input logic reset,
input logic set,
input logic [3:0] d,
output logic [3:0] q);

```

        always_ff @(posedge clk, posedge reset)
            if (reset) q <= 0;
            else      q <= d;
        always @(set)
            if (set)  q <= 1;
    endmodule

```

- (j) module and3(input logic a, b, c,
output logic y);

```

    logic tmp;
    always @(a, b, c)
    begin
        tmp <= a & b;
        y  <= tmp & c;
    end
endmodule

```

VHDL Exercises

The following exercises are specific to VHDL.

Exercise 4.51 In VHDL, why is it necessary to write

```

q <= '1' when state = S0 else '0';

```

rather than simply

```

q <= (state = S0);

```

Exercise 4.52 Each of the following VHDL modules contain errors. For brevity, only the architecture is shown; assume that the library use clause and entity declaration are correct. Explain the errors and show how to fix them.

- (a) architecture synth of latch is
begin

```

    process(clk) begin
        if clk = '1' then q <= d;
        end if;
    end process;
end;

```

- (b) architecture proc of gates is
begin

```

    process(a) begin
        Y1 <= a and b;
        y2 <= a or b;
        y3 <= a xor b;
        y4 <= a nand b;
        y5 <= a nor b;
    end process;
end;

```

- (c) architecture synth of flop is
- ```
begin
 process(clk)
 if rising_edge(clk) then
 q <= d;
 end if;
 end process;
```
- (d) architecture synth of priority is
- ```
begin
  process(all) begin
    if a(3) then y <= "1000";
    elsif a(2) then y <= "0100";
    elsif a(1) then y <= "0010";
    elsif a(0) then y <= "0001";
    end if;
  end process;
end;
```
- (e) architecture synth of divideby3FSM is
- ```
type statetype is (S0, S1, S2);
signal state, nextstate: statetype;
begin
 process(clk, reset) begin
 if reset then state <= S0;
 elsif rising_edge(clk) then
 state <= nextstate;
 end if;
 end process;

 process(state) begin
 case state is
 when S0 => nextstate <= S1;
 when S1 => nextstate <= S2;
 when S2 => nextstate <= S0;
 end case;
 end process;

 q <= '1' when state = S0 else '0';
end;
```
- (f) architecture struct of mux2 is
- ```
component tristate
  port(a: in STD_LOGIC_VECTOR(3 downto 0);
       en: in STD_LOGIC;
       y: out STD_LOGIC_VECTOR(3 downto 0));
end component;

begin
  t0: tristate port map(d0, s, y);
  t1: tristate port map(d1, s, y);
end;
```

```
(g) architecture asynchronous of floprs is
begin
  process(clk, reset) begin
    if reset then
      q <= '0';
    elsif rising_edge(clk) then
      q <= d;
    end if;
  end process;

  process(set) begin
    if set then
      q <= '1';
    end if;
  end process;
end;
```


Interview Questions

The following exercises present questions that have been asked at interviews for digital design jobs.

Question 4.1 Write a line of HDL code that gates a 32-bit bus called `data` with another signal called `sel` to produce a 32-bit `result`. If `sel` is `TRUE`, `result` = `data`. Otherwise, `result` should be all 0's.

Question 4.2 Explain the difference between blocking and nonblocking assignments in SystemVerilog. Give examples.

Question 4.3 What does the following SystemVerilog statement do?

```
assign result = |(data[15:0] & 16'hC820);
```

